ITCS213 Computer Organization and Architecture

Class: 1-2025
Sec 2 Wed 9:00-12:00 IT311
Sec 3 Thu 9:00-12:00 IT311
Sec 1 Thu13:00-16:00 IT311

Unit 4: **ARM MUL** and **DIV**, **Data Hazards**, Assembly on **MIPS**

Lecturer: Dr. Peter M.D. Scully
Faculty of ICT, Mahidol University
peter.scu@mahidol.edu

Slides adapted from Subject Coordinator:
Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Announcement: ITCS213

Unit 4: ARM MUL and DIV, Data Hazards, Assembly on MIPS

Announcements

- **Homework:** Unit 4 Assignment (*MIPS assembly coding*) Lab 1.

Today we aim to:

1. *Learn why MUL and DIV can be slow, or fast:*
   a. *Understand Division.*
   b. *Understand Multiplication.*
   c. *The importance of ADD and SUB.*

2. *How Pipelined Instructions speed-up execution.*
   a. *But what can go wrong (slow-downs),*
      - *types of conflicts (Data Hazards),*
      - *how they are solved (Stall, Forward, Flush).*

3. *Understanding Overflows and Carry bits.*

Agenda

Part A: Reminder : Pipeline Architectures

Part B: MUL Multiplication (ARM)

Part C: DIV Division (ARM)

Part D: Data Hazards in Pipelined Processors

Part E: Pipeline Hazards

Part F: Data Hazards Handling

Part G: Assembly Programming on MIPS

Computer Organization Study Process:

How we build understanding — from pipeline architecture to assembly hazards

Step 1: Pipeline Fundamentals

ISA pipeline stages, hardware operations

Step 2: Multiplication & Division

MUL/DIV on ARM, shift-and-add algorithms

Step 3: Data Hazards

RAW, WAR, WAW, RAR dependencies in pipelines

Step 4: Hazard Solutions

Stall insertion, data forwarding, flushing

Step 5: MIPS Assembly

Registers, R-type instructions, arithmetic

Step 6: Overflow & Carry

Signed/unsigned range, 2's complement

All steps are covered in this unit (Unit 4).

Grading

ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

GRADING SYSTEM				
Grade Criteria		A	>=	80
		B+	>=	75
		B	>=	70
		C+	>=	65
		C	>=	60
		D+	>=	55
		D	>=	50
		F	<	50

Content

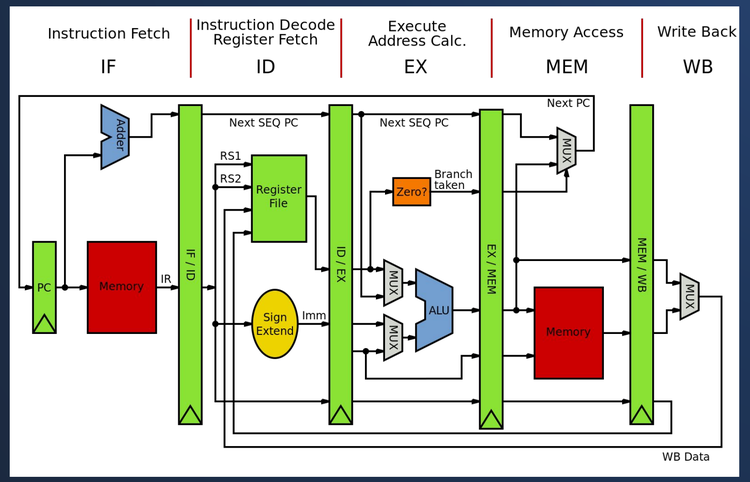
- **MUL** and **DIV** instructions
- **Pipelining**
- **Data Hazards** in **Pipelined** Processors
- Assembly Programming on MIPS (LAB Exercise)

PART

Types of Pipelining:

("Look ahead instructions" or staggered instruction sequences)

Arithmetic and instruction pipelining — two ways to overlap work.





	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

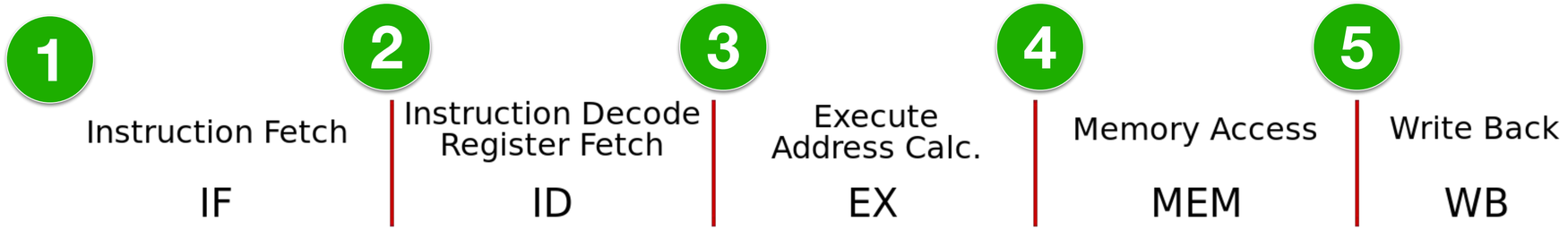
ISAs



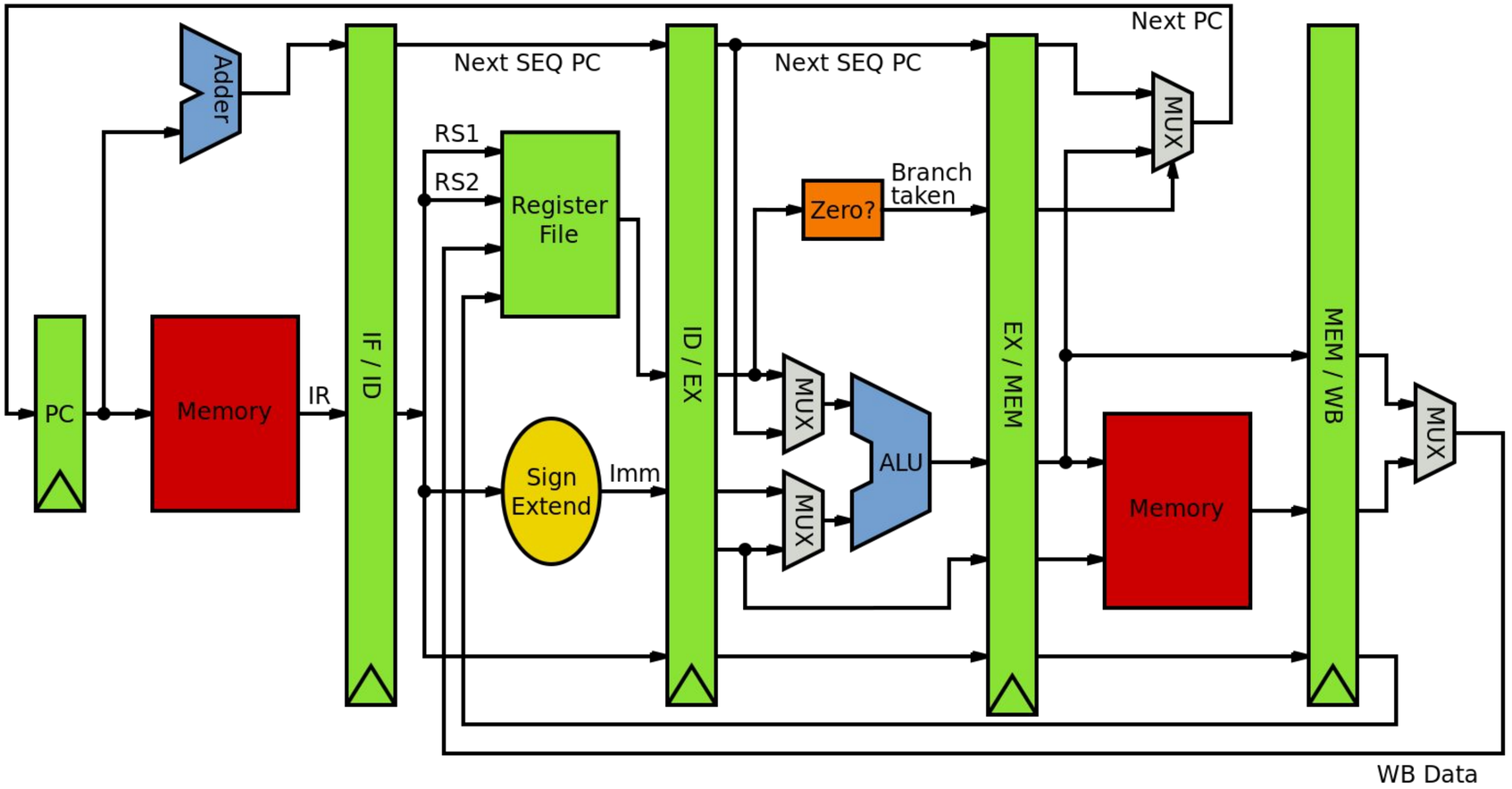
- Design Philosophy: RISC vs CISC

	ARM	MIPS	x86
Type	RISC (Reduced Instruction Set Computer)	RISC	CISC (Complex Instruction Set Computer)
Goal	Fewer, simpler instructions that execute quickly	Same as ARM — simple, load/store model	Many complex instructions, some doing multiple steps in one opcode
Instruction Length	Fixed (usually 32-bit, some 16-bit Thumb)	Fixed 32-bit	Variable (1–15 bytes per instruction)
Registers	16 general-purpose (32-bit in ARMv7, 64-bit in ARMv8)	32 general-purpose	Few general-purpose (x86-64 has 16, but original x86 had only 8)

Pipeline Stages of **Instruction Set** Architectures: **MIPS**



5-stage
Pipeline

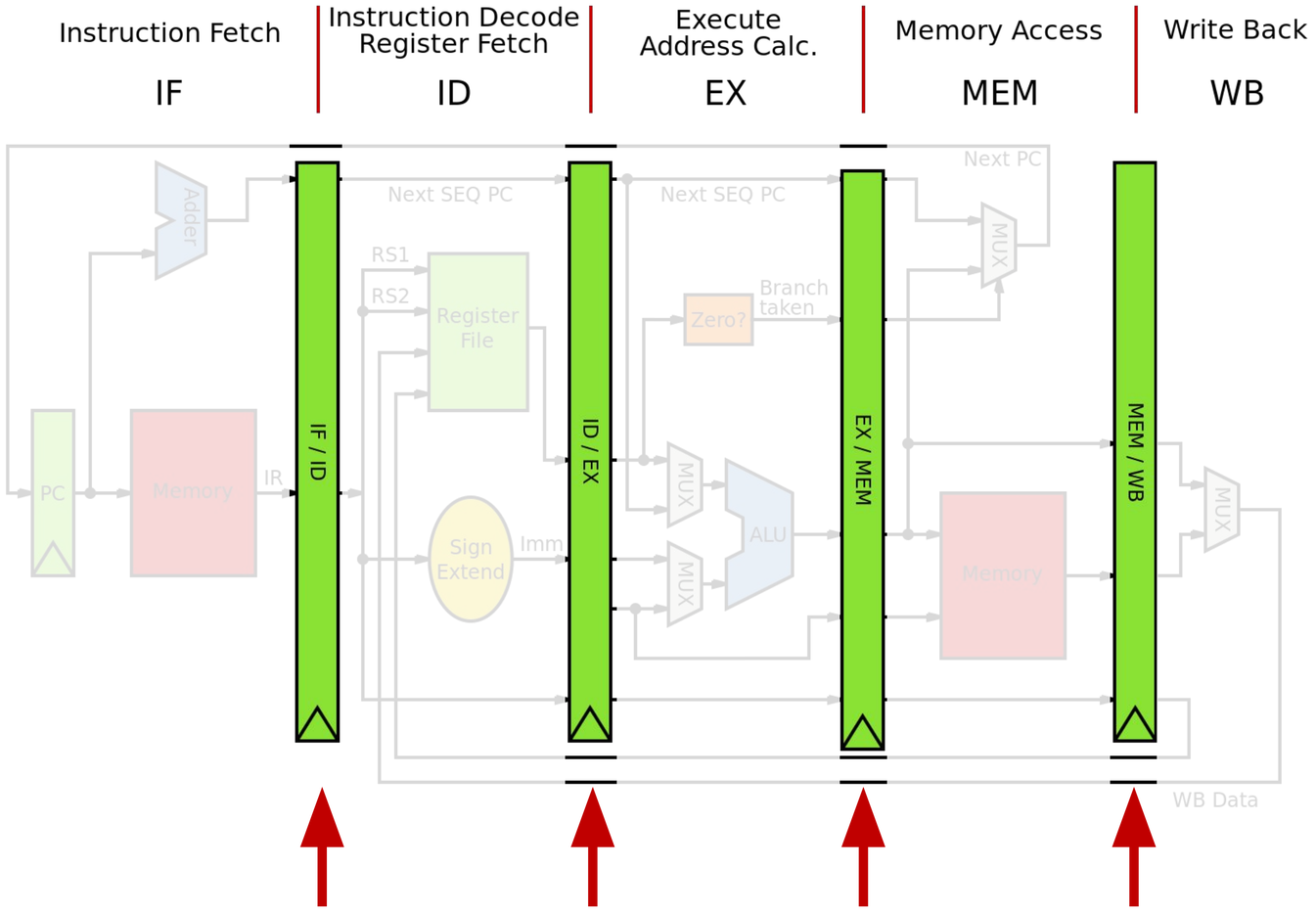


Stages:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Memory Access
5. Write Back

Pipeline Stages of **Instruction Set** Architectures: **MIPS**

5-stage **Pipeline**



Pipeline Registers

(vertical green bars)

They store the intermediate data and control signals between stages.

- **IF/ID** – Between IF and ID
- **ID/EX** – Between ID and EX
- **EX/MEM** – Between EX and MEM
- **MEM/WB** – Between MEM and WB

Allow **multiple instructions** to be in **different stages at the same time.**

Types of Pipelining

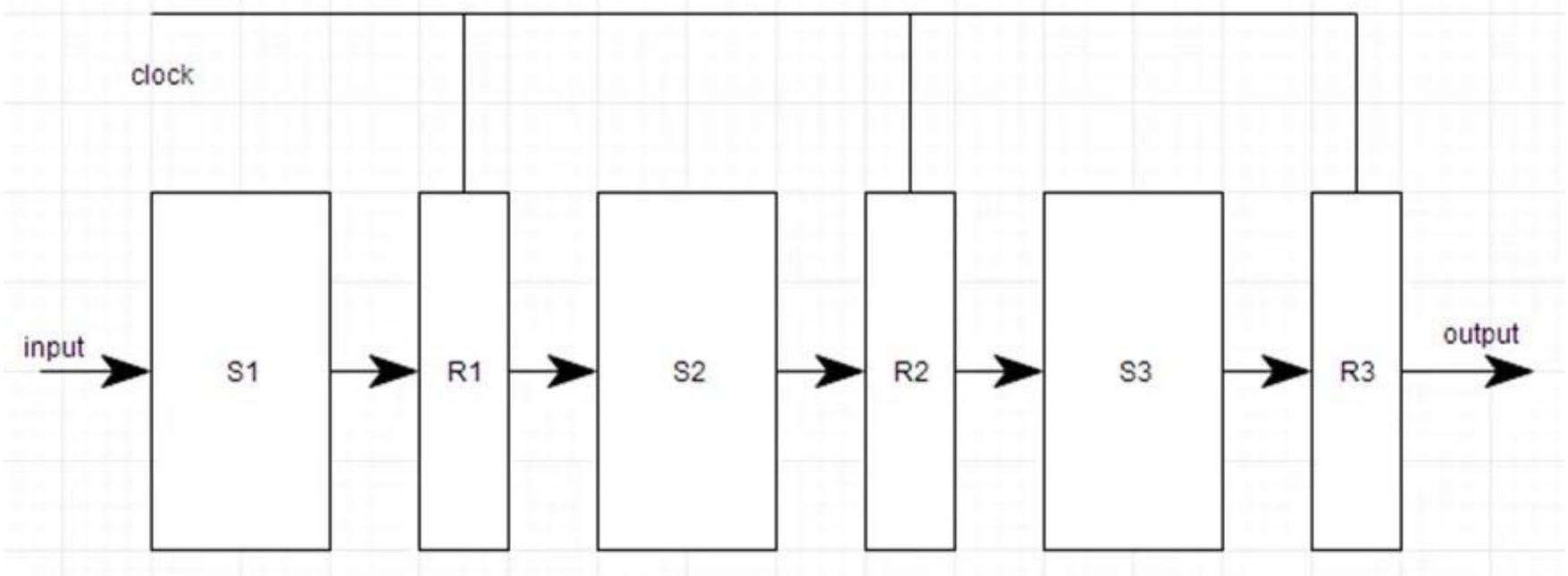
- In 1977, Handler and Ramamoorthy separated pipelining processors according to their performance.
  - **Arithmetic Pipelining**
  - **Instruction Pipelining**

Arithmetic Pipelining

- Arithmetic **Pipelining** is designed to calculate the addition, multiplication, and division of a floating-point number at high speed.
- Many arithmetic logic units are built into the system to perform parallel arithmetic calculations in a variety of data formats.
  - Examples of **arithmetic** pipelined processors:
    - Star-100
    - TI-ASC
    - Cray-1
    - Cyber-205

Instruction Pipelining

- A **number** of **instructions are pipelined** (*queued*), and the execution of the current instruction overlaps with the execution of the next instruction.
- It is also known as instruction lookahead.





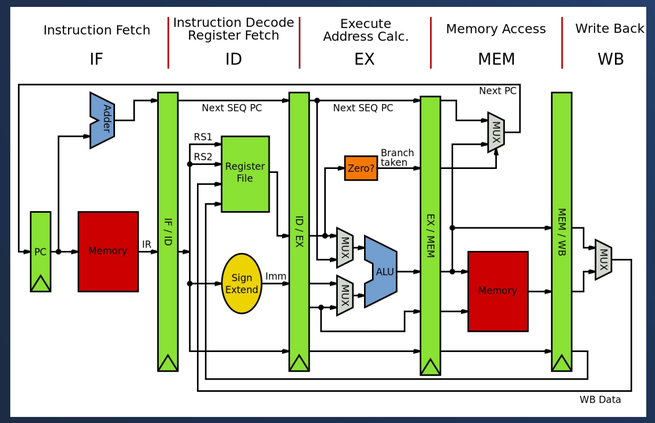
Key Points: Types of Pipelining

- **Arithmetic pipelining speeds up FP operations**
- **Instruction pipelining overlaps fetch/decode/execute stages**
- **Both types improve throughput without faster clock speed** *(duty cycle).*

PART

Pipelined Processors

How the five-stage pipeline executes many instructions at once.

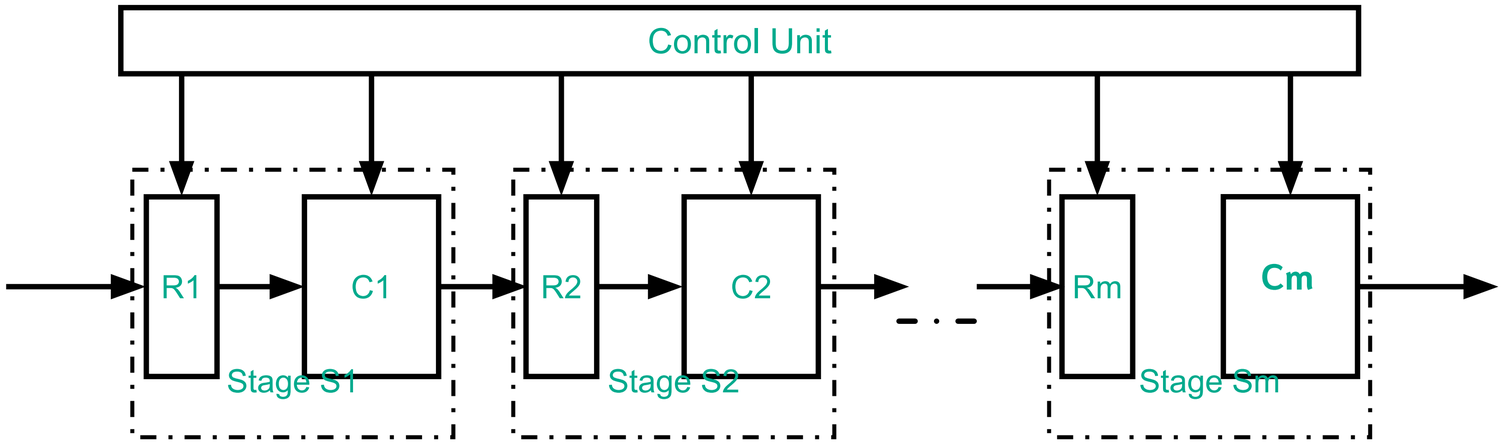




	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

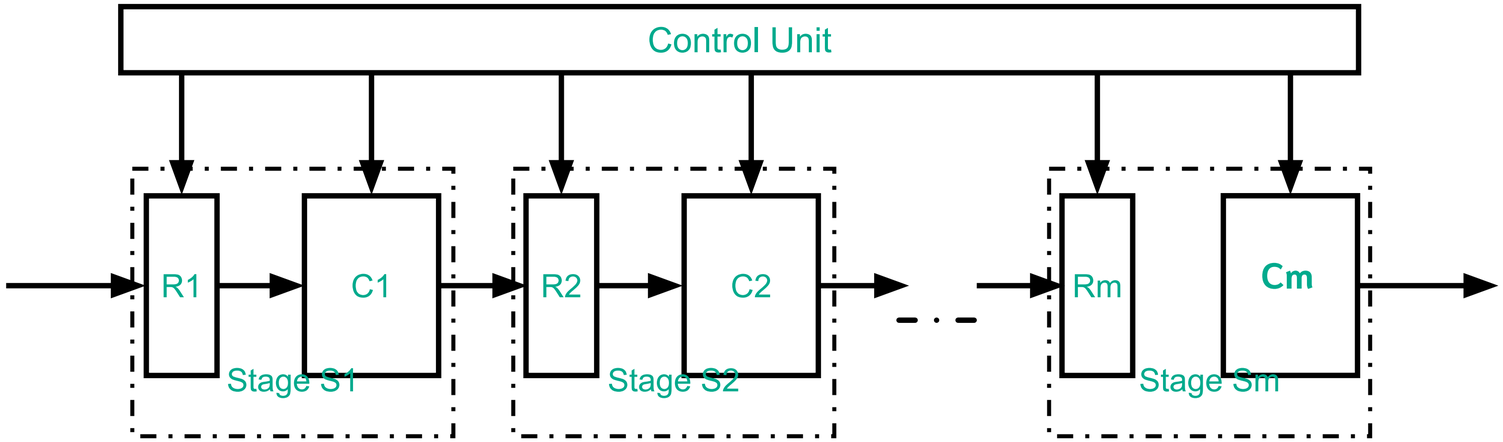
Pipelining

- **Pipelining** is the process of collecting instructions from a processor using a **pipeline**.
- **Pipelining** allows us to save and execute instructions through a systematic process.
- **Pipelining** is also called **pipeline** processing.



Pipelining

- Pipe installation is the method by which many instructions are concealed during execution.
- The pipe is divided into sections connected to form a pipe-like structure.
- Commands come in and out of another.
- Instructions come in from one end of the pipe and out of another end.
- It increases the total instruction throughput.



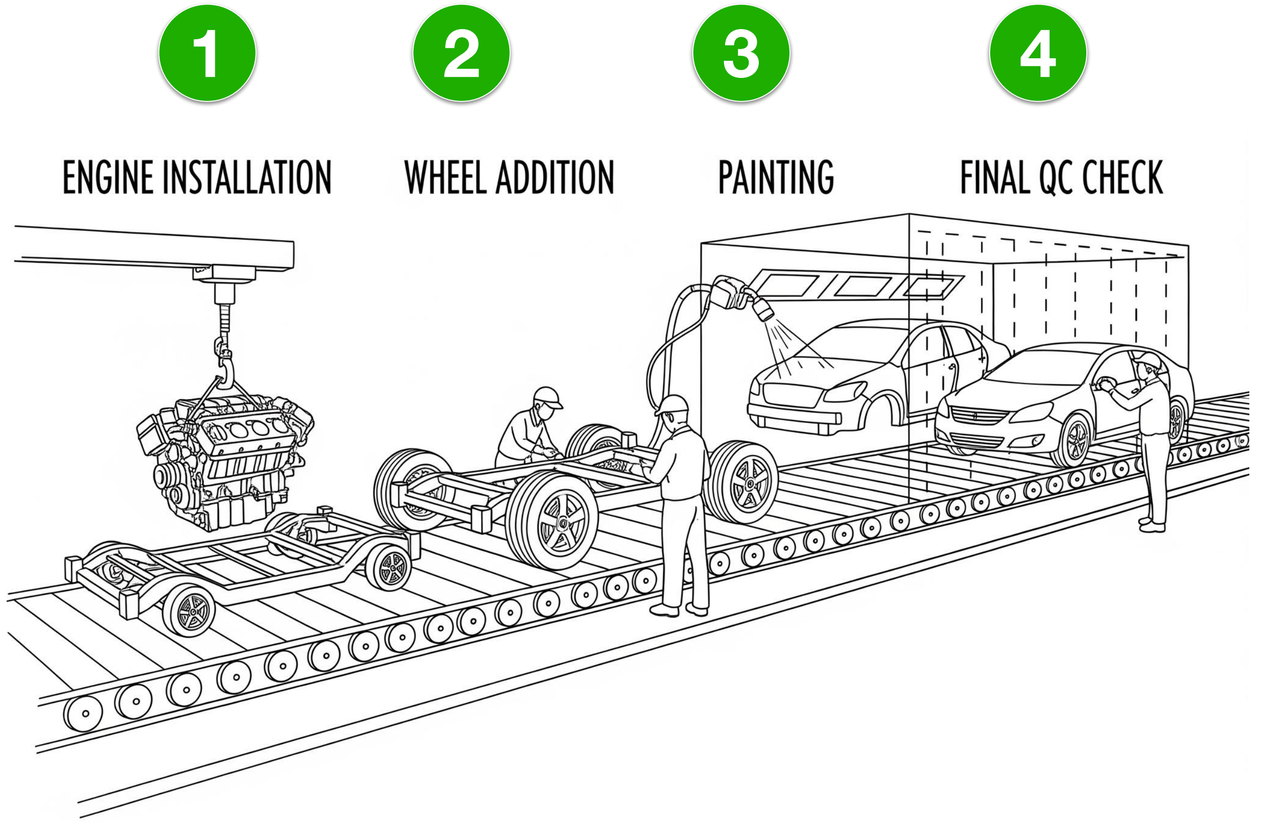
Pipelining

- While one instruction is being executed, others are moving through the earlier stages, allowing multiple instructions to be processed at the same time.

- This is similar to an assembly line in a factory,
  - where different workers (stages) handle different tasks on multiple products (instructions) simultaneously.

Pipeline Processor

- An **assembly line in a factory**, for example, consider how cars are built in a factory:
  - One worker installs the **engine**.
  - The next one adds the **wheels**.
  - Another **paints** the car.
  - The last one performs **final checks**.

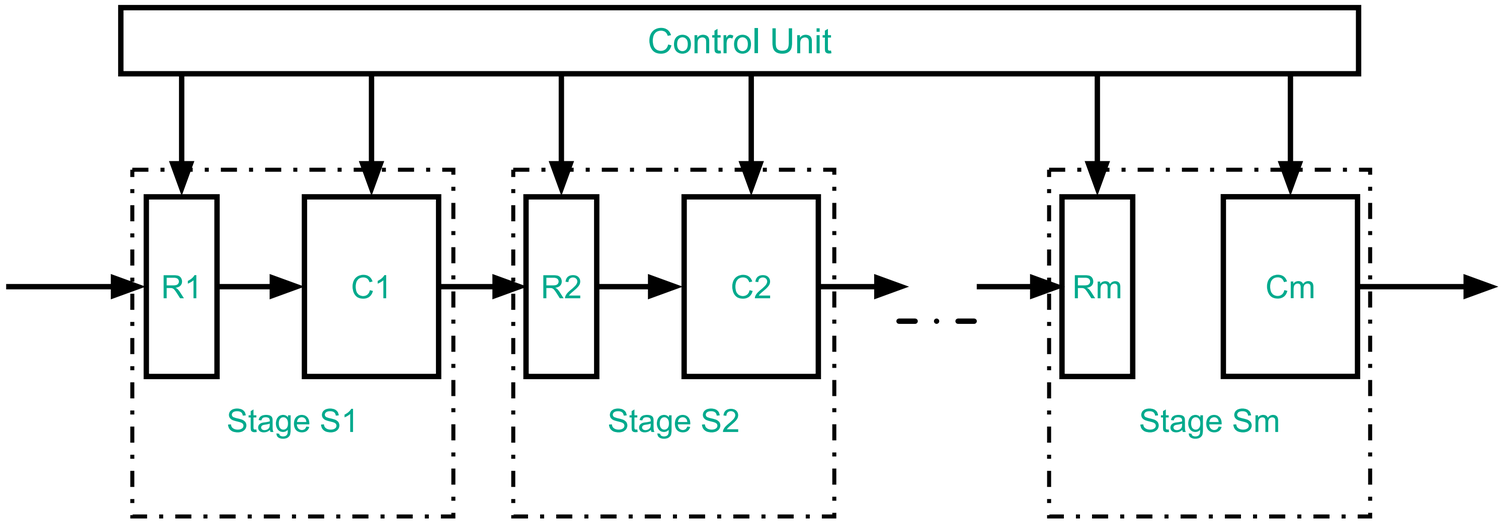


(While the next car chassis is waiting to start)

Pipeline Processor

- This overlap **increases** the overall **efficiency** and **throughput** of the processor.

- In **clock cycle 1**, data enters Stage S1 (R1 → C1).
- In **clock cycle 2**, the data moves to Stage S2 (R2 → C2), while new data enters Stage S1.



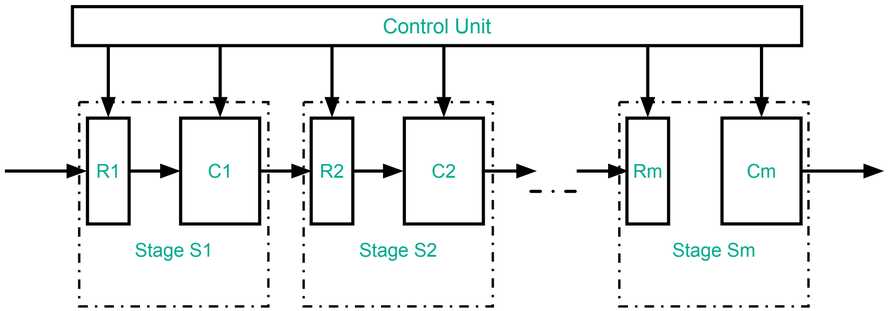
Pipeline Processor

- This process continues, and each stage is **simultaneously** **processing a different piece of data**.

- Eventually, the output appears at the end after passing through all stages.

Clock Cycle	Stage S1 (R1→C1)	Stage S2 (R2→C2)	Stage S3 (R3→C3)	...	Stage Sm (Rm→Cm)
1	Data A	-	-	...	...
2	Data B	Data A	-	...	...
3	Data C	Data B	Data A	...	...
4	Data D	Data C	Data B	...	...
5	Data E	Data D	Data C	...	...
...	...	...	...	...	...

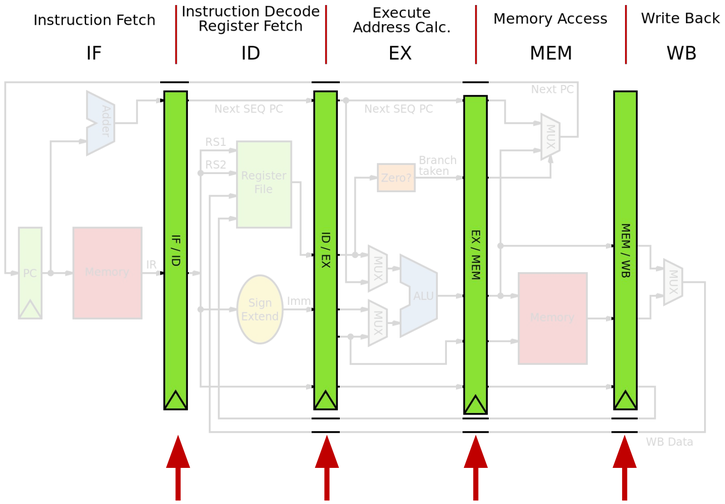
Pipelining



- In the **pipeline** system, each segment contains:
  - an **input register**
  - followed by a **combinational circuit**.

- The register holds data, and the combinational circuit performs functions on it.

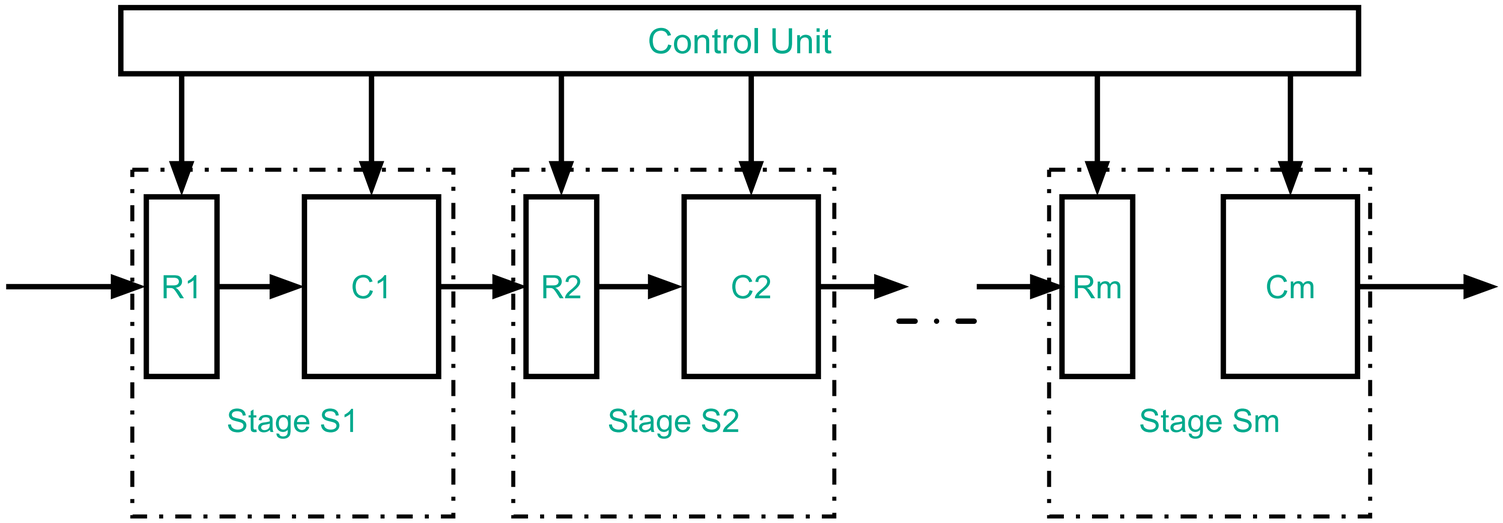
- The output of the combinational circuit is used in the input register of the next segment.



Pipeline Registers
(vertical green bars)

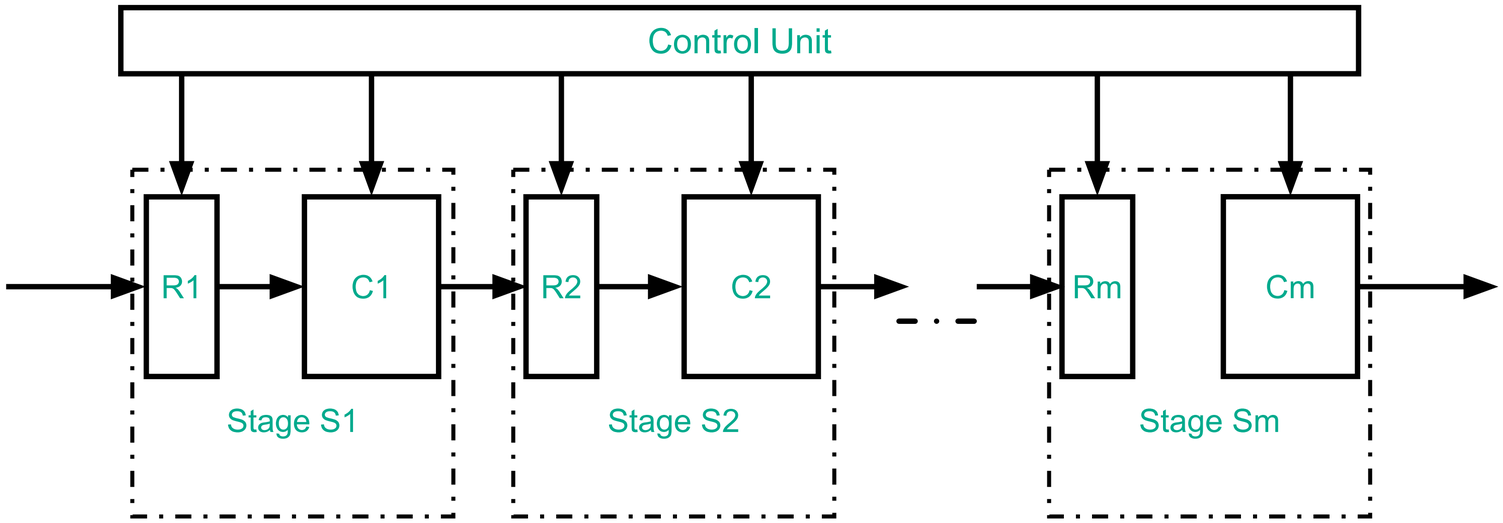
Pipelining: Components in the Diagram

- **Data In:**
  - This is the input data that enters the **pipeline**.
- **Stages (S1, S2, ..., Sm):** Each stage performs part of the operation.
  - The **pipeline** is divided into multiple stages (S1, S2, ..., Sm), and each stage handles a specific operation.
- **Data Out:** The final output after processing is complete across all **pipeline** stages.



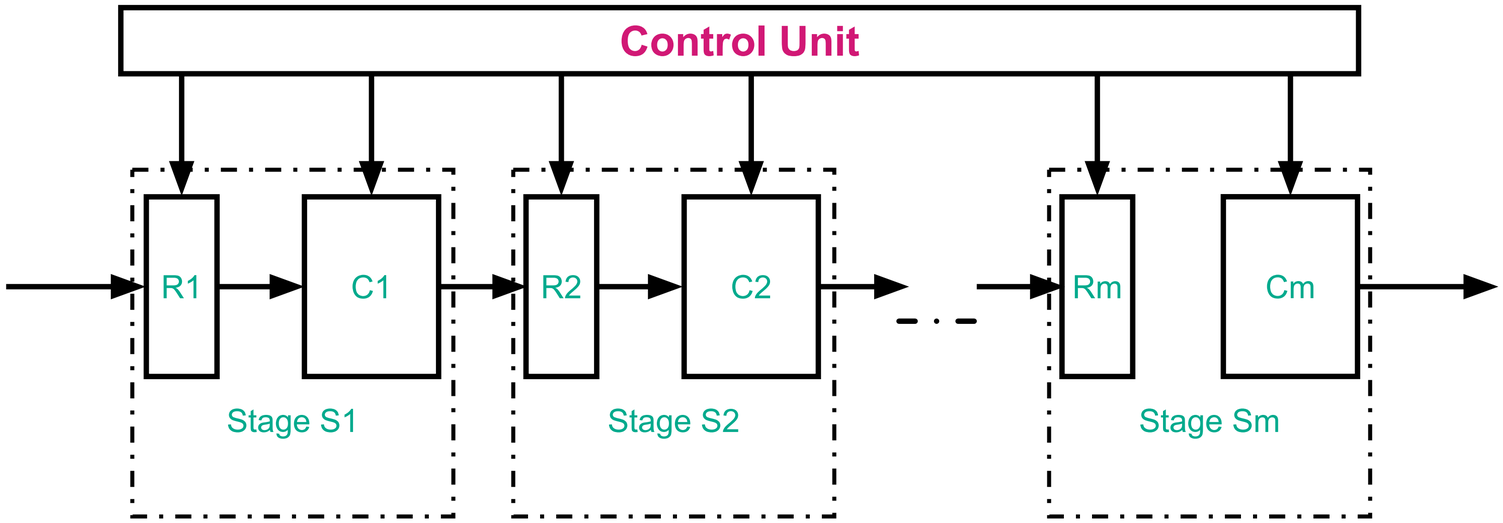
Pipelining: Components in the Diagram

- Registers **(R1, R2, ..., Rm):** These are **pipeline** registers.
  - They temporarily hold data between stages to ensure smooth transfer and isolation between operations.
  - Example: R1 stores the output of Stage S1 before passing it to S2.



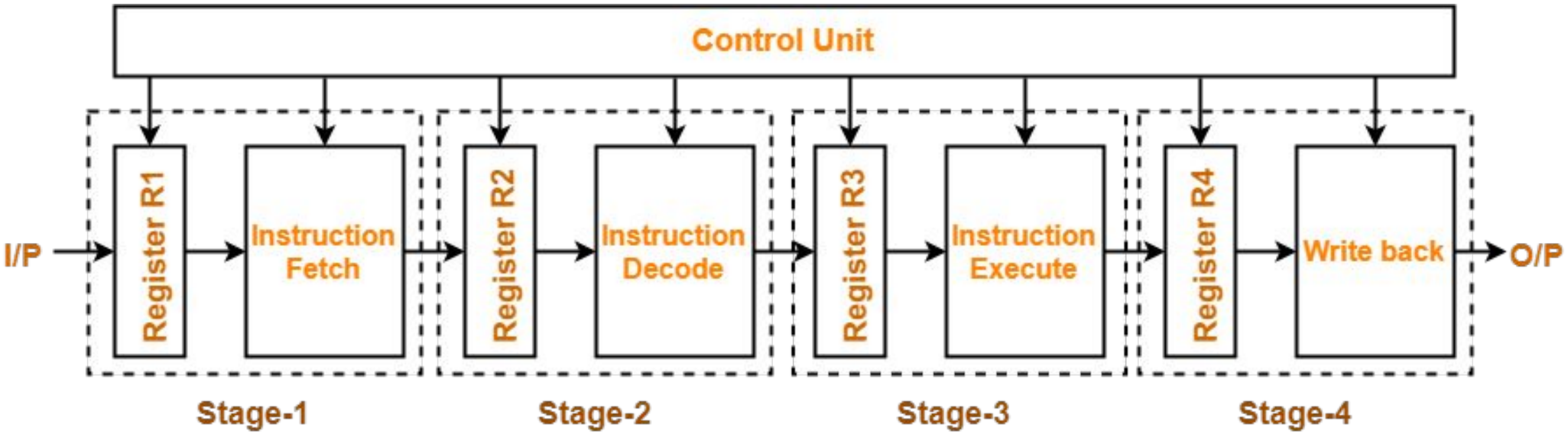
Pipelining: Components in the Diagram

- **Computation Units (C1, C2, ..., Cm):**
  - These perform the actual processing (like arithmetic or logical operations by **ALU**).
    - *Each computation unit corresponds to a specific stage. (e.g. IF, ID, EX, MEM, WB)*
- **Control Unit:** Manages the timing and control signals for each stage.
  - Ensures that each stage **operates in sync** and processes the correct data at the right time.



4-stage Pipelined Executions

4-stage
Pipeline



Four Stage Instruction Pipeline

Stages:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute ~~*Memory Access*~~
4. Write Back

4-Stage removes ~~Memory Access~~ stage (Find out why in next section)

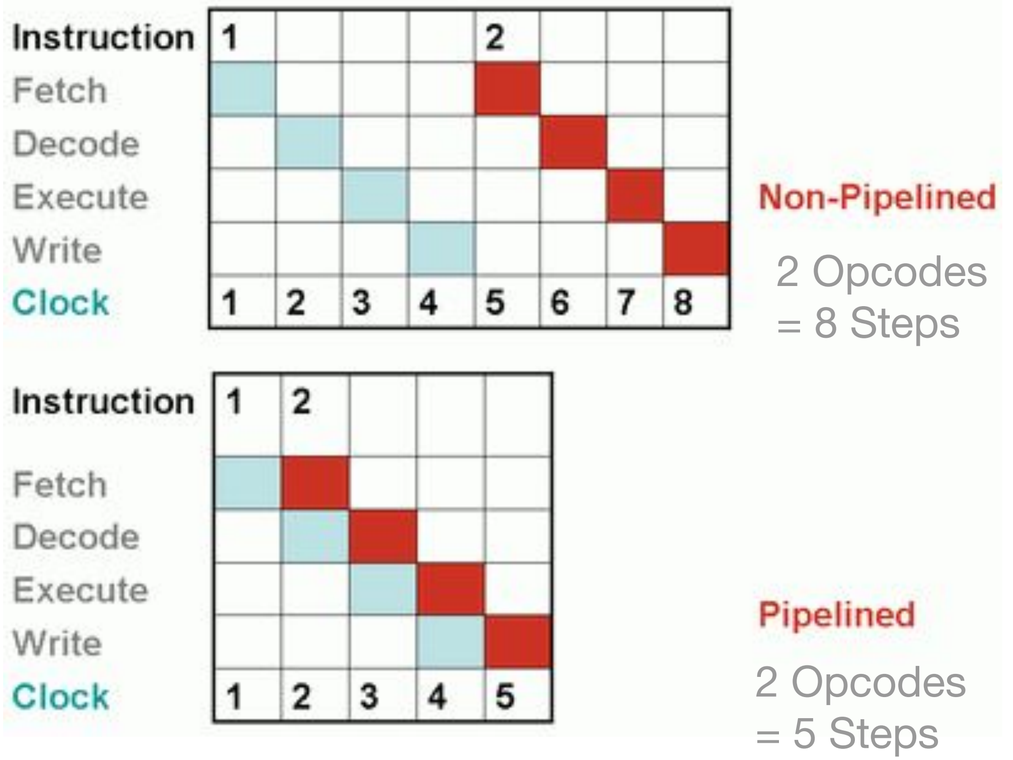
source: https://www.gatevidyalay.com/pipelining-in-computer-architecture/

Pipeline Processor:
Non-Pipelined vs. Pipelined Executions

- A **pipeline processor** is a type of CPU architecture that improves processing speed by dividing instruction execution into separate stages.
  - Timing differs and throughput wall-clock efficiency differs.

- Each stage handles a specific part of the instruction, such as *fetching*, *decoding*, *executing*, **memory access**, and *writing* results.

4-stage **Instructions Example**

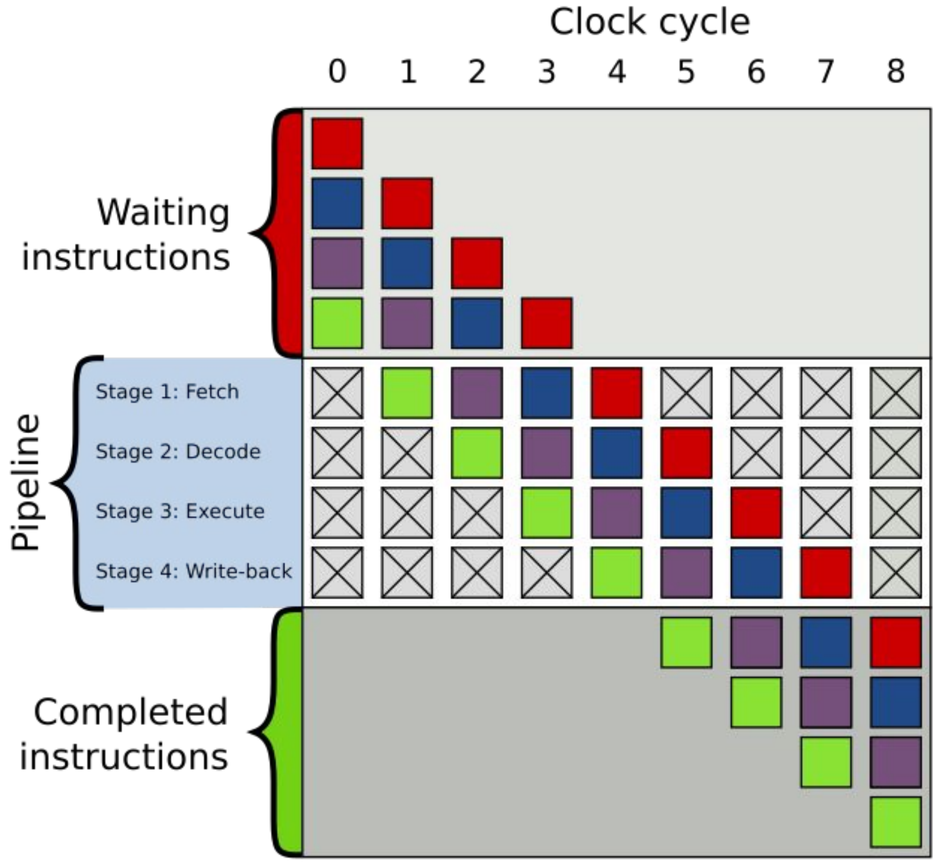


Also 4-Stage removes ~~Memory Access~~ stage.

Basic 4-stage Pipeline

- The following is an execution of the same four instructions through a 4-stage **pipeline**.





source: https://en.wikipedia.org/wiki/Pipeline_stall

Basic 5-stage Pipeline

- The following is an execution of the same four instructions through a 5-stage **pipeline**.

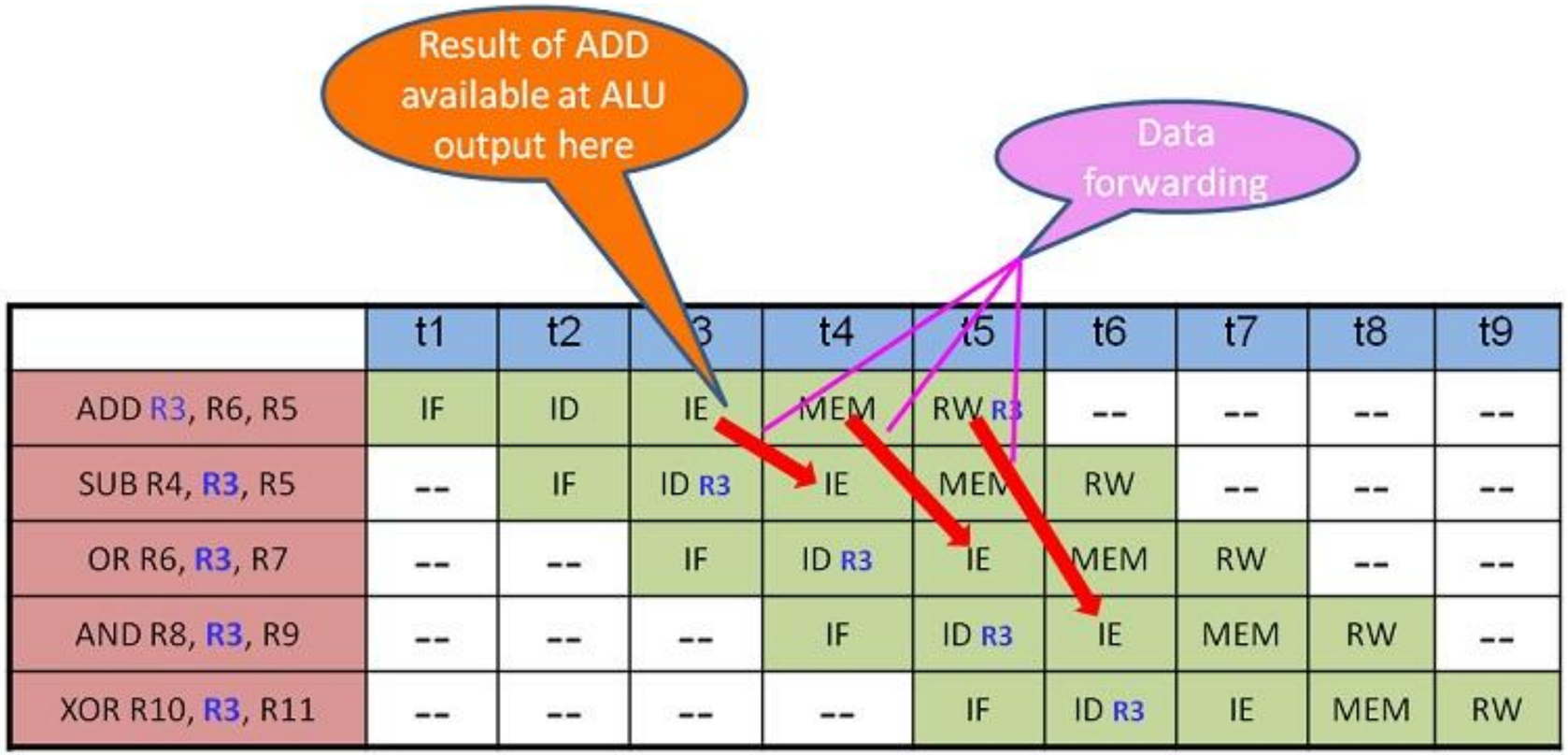
Instr. No. \ Clock cycle	1	2	3	4	5	6	7
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX

Where's the ALU

source: https://en.wikipedia.org/wiki/Instruction_pipelining

Basic 5-stage Pipeline

- The following is an execution of the same four instructions through a 5-stage **pipeline**.



	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

IE / EX:
Instruction Execution

source: https://en.wikipedia.org/wiki/Instruction_pipelining



Key Points: Pipelined Processors

- **Pipeline divides instruction execution into stages**
  - IF, ID, EX, MEM, WB in classic 5-stage RISC pipeline
- **Pipeline registers store intermediate data between stages**
  - IF/ID, ID/EX, EX/MEM, MEM/WB register banks
- **Throughput improves but individual instruction latency unchanged**

PART A

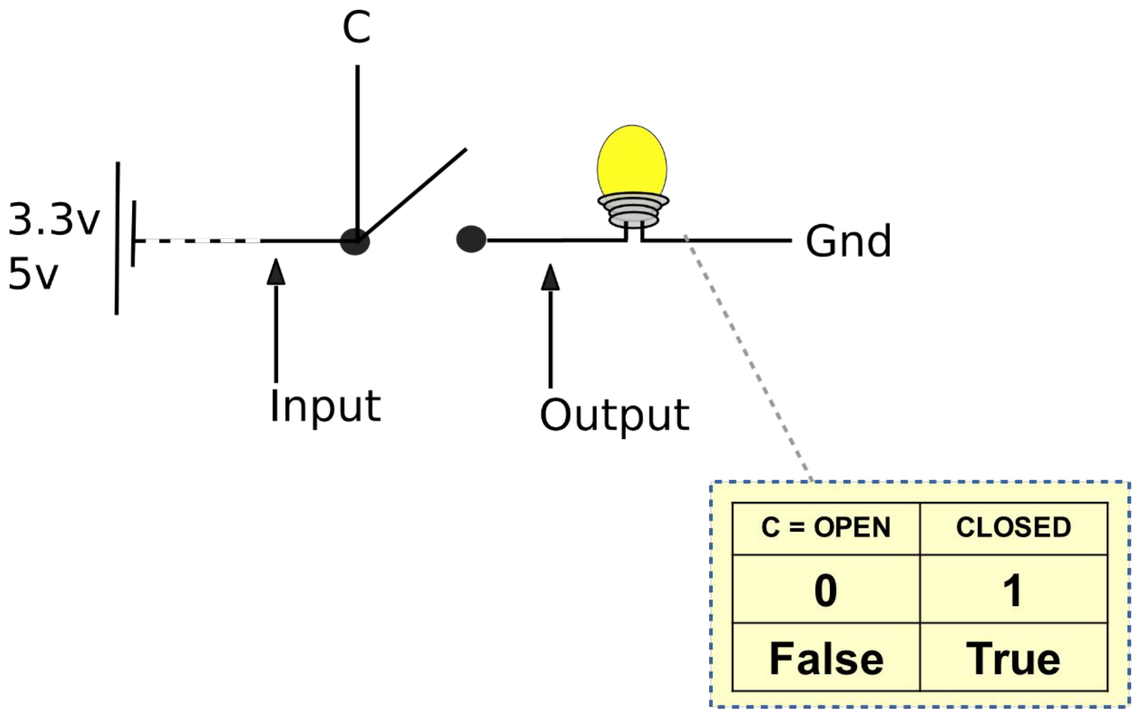
Refresher: **On Logic Gates and Making Arithmetic / Logic Circuits**

How instruction set architectures map onto pipelined hardware.
How MUL , DIV are related to ADD , SUB

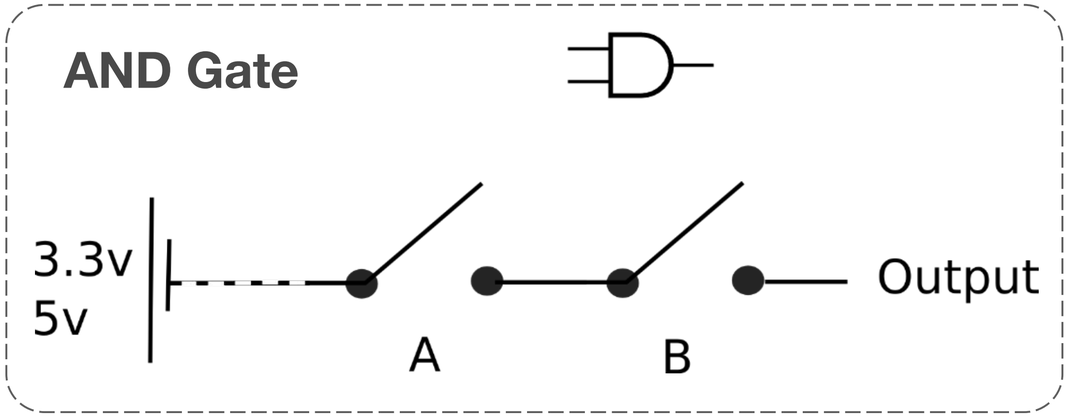
Hardware Operations: Logic Gates

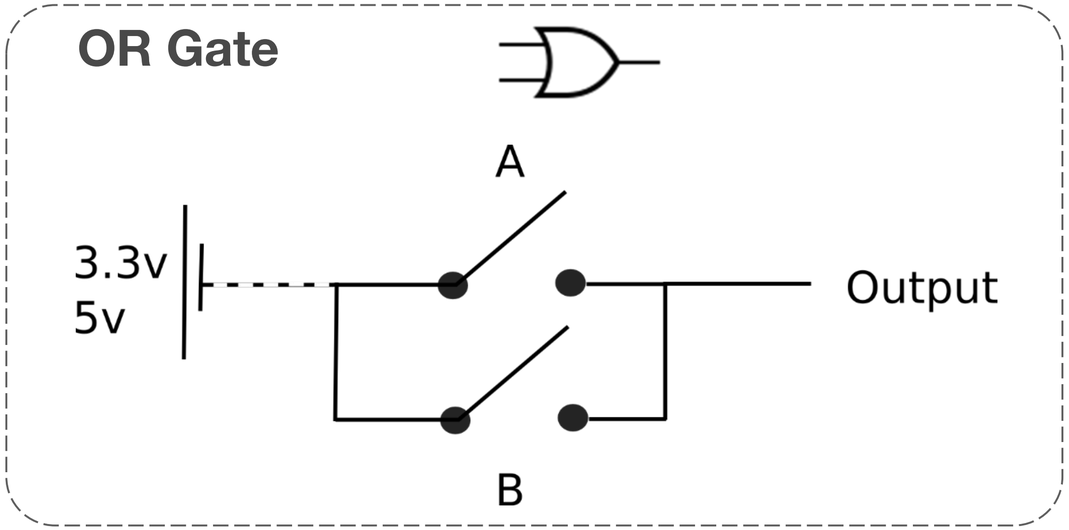


- **All ALU circuits are made from logic gates**.
- Logic gates are basically, **circuits with open or closed switches**.
  - A **memory bit** (C) **will** either **open or close** a **switch**, to control the logic gate.

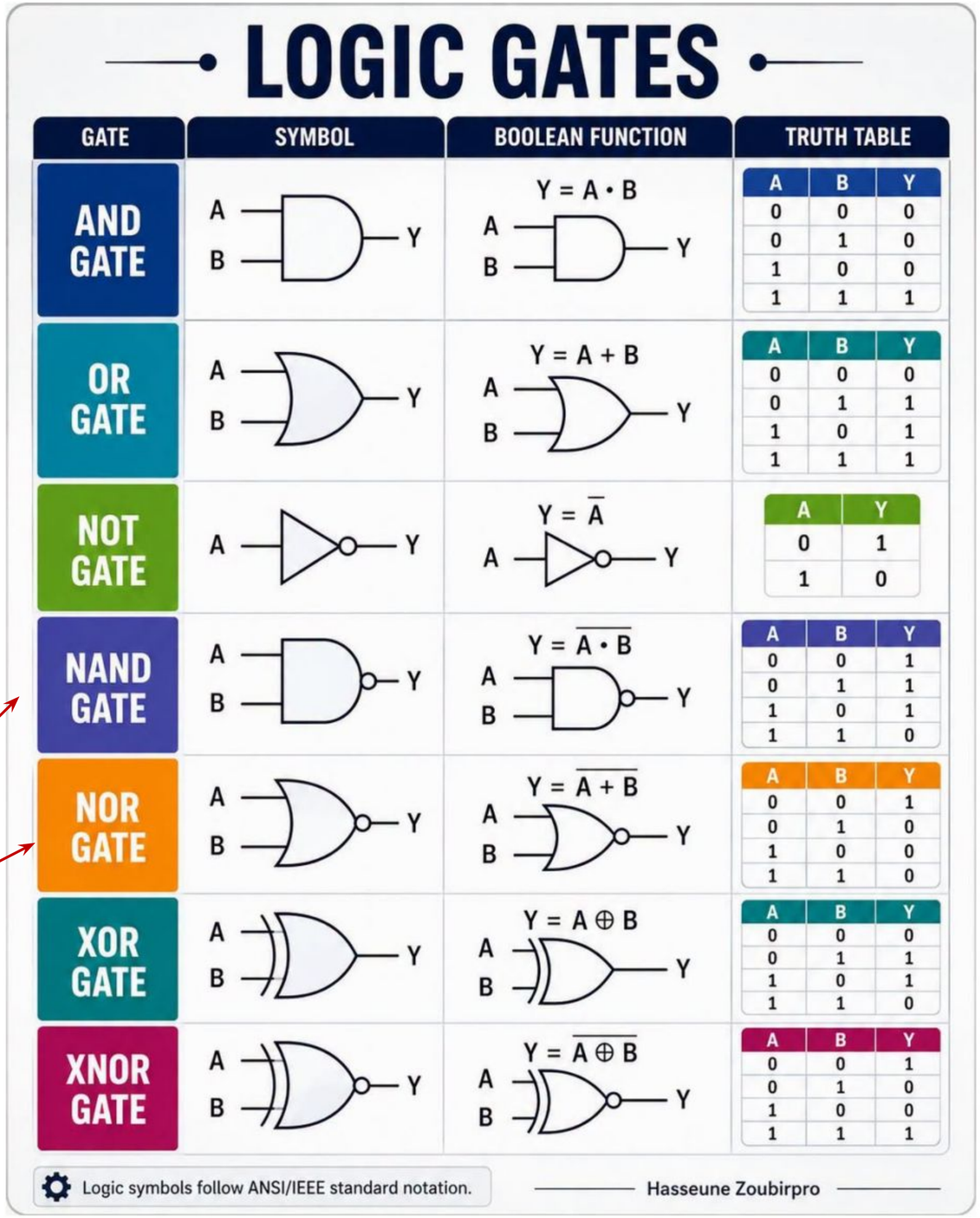


C = OPEN	CLOSED
0	1
False	True





Hardware Operations:
Logic Gates & Truth Tables



Universal Gates: Can build all other gates with these.

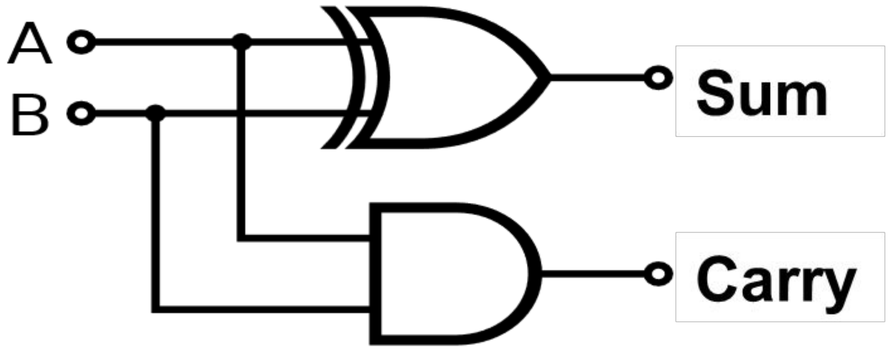
Hardware Operations: ADD and SUB

- ADD and SUB are fundamental building blocks.
- They are simple, fast, and implemented directly in hardware logic gates.
- Every CPU must have these.

Hardware Operations: ADD



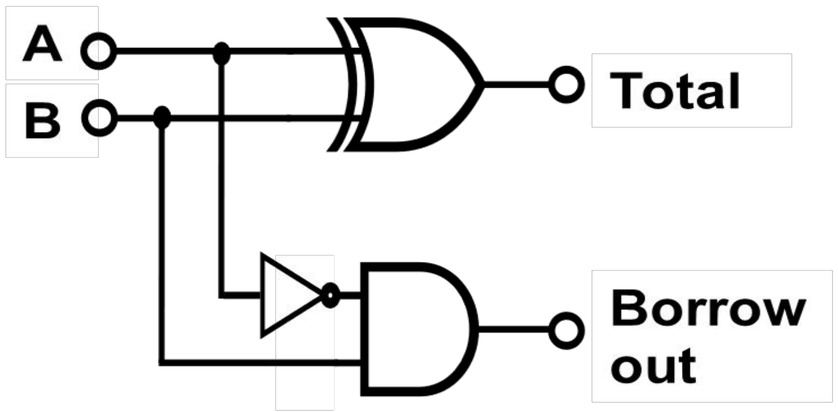
1-Bit Binary Addition Circuit
(The Half-Adder)



Input		Output	
A	**B**	**Sum (XOR)**	**Carry (AND)**
0	0	**0**	**0**
1	0	**1**	**0**
0	1	**1**	**0**
1	1	**0**	**1**

Hardware Operations: SUB

1-Bit Binary Subtraction Circuit
(The Half-Subtractor)



Input		Output	
A	B	Total XOR	Borrow out (NOT A) AND B
0	0	**0**	**0**
1	0	**1**	**0**
0	1	**1**	**1**
1	1	**0**	**0**

Hardware Operations: **MUL** and **DIV** are complex

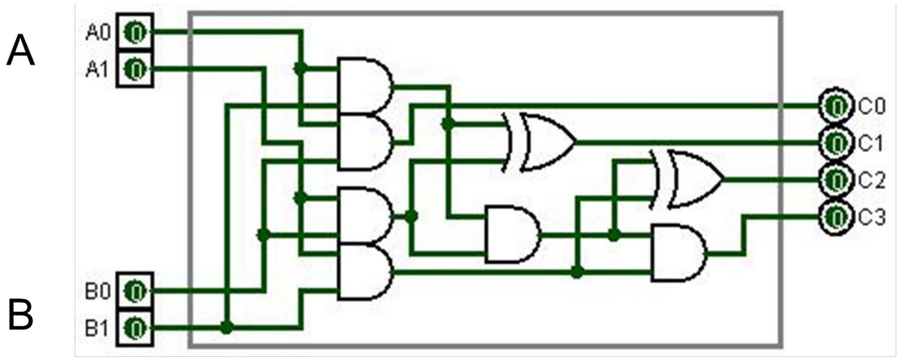
- **MUL** and **DIV** are complex, multi-step operations.
  - MUL actually uses ADD (repeatedly)
  - DIV actually uses SUB (repeatedly)

Example of **2-bit Multiply** (right):

- much more complex to build, many steps.



Binary Multiplication Circuit
(2-bit Multiplier)



- repeated Half-Adder circuit. (two adds)

Hardware Operations: **MUL** and **DIV** are complex

- A CPU designer has a choice:
  - ✅**Do build** dedicated hardware for them:
    - making the CPU bigger, more expensive, and using more power, but much faster.
  - ❌**Don't build** the hardware and let the compiler/software figure it out using **loops of simpler instructions**:
    - making the CPU smaller/cheaper, but these operations are slower.

Hardware Operations

- Early ARM CPUs, like the ARM7TDMI, chose option 2 for **DIV** and a middle ground for **MUL**.

- Modern ARMs, such as the Cortex-A series, Cortex-M3, and above, have hardware for both.



Key Points: ISAs on Pipeline Architecture

- **ADD and SUB: fundamental operations in hardware logic gates**
  - Simple, fast, single-cycle execution
- **MUL and DIV: complex operations on simple CPUs**
  - Require multi-cycle algorithms or dedicated hardware
- **ARM CPU evolution for MUL/DIV support**
  - Early (ARM7TDMI): software DIV, partial hardware MUL
  - Modern (Cortex-A, M3+): dedicated hardware for both

PART

MUL Multiplication on a Simple CPU (ARM)

Multiplying on a simple CPU: the ARM MUL instruction in action.

MUL Multiplication

- Multiplication (MUL) on a simple CPU
  - Even if a CPU has **MUL** instructions, it's instructive to see how it could be done with a loop.
  - The classic algorithm is shift-and-add.

- Multiplication is just repeated addition:

  5 * 3 is 5 + 5 + 5

- But this is slow for large numbers.
- The shift-and-add method is far more efficient, leveraging binary.

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  1. Initialize a Result register to 0.
  2. Initialize a Counter to 32 (the number of bits).
  3. Look at the Least Significant Bit (LSB) of the Multiplier.
  4. If the LSB is 1, add the Multiplicand to the Result.
  5. Shift the Multiplicand left by 1 bit (doubling it).
  6. Shift the Multiplier right by 1 bit (so we can check the next bit).
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.

MUL Multiplication

- ARM Assembly Pseudocode (if **MUL** didn't exist) for R0 = R1 * R2:

- Algorithm (for two 32-bit numbers):
  1. Initialize a Result register to 0.
  2. Initialize a Counter to 32 (the number of bits).
  3. Look at the Least Significant Bit (LSB) of the Multiplier.
  4. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21   @ Multiplicand = 21
  MOV R2, #10   @ Multiplier = 10
  MOV R0, #0    @ Result = 0
  MOV R3, #32   @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2      @ Check the LSB of the Multiplier (R2)
  BEQ  no_add      @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  1. Initialize a Result register to 0.
  2. Initialize a Counter to 32 (the number of bits).
  3. Look at the Least Significant Bit (LSB) of the Multiplier.
  4. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21   @ Multiplicand = 21
  MOV R2, #10   @ Multiplier = 10
  MOV R0, #0    @ Result = 0
  MOV R3, #32   @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2      @ Check the LSB of the Multiplier (R2)
  BEQ  no_add      @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  1. Initialize a Result register to 0.
  2. Initialize a Counter to 32 (the number of bits).
  3. Look at the Least Significant Bit (LSB) of the Multiplier.
  4. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21   @ Multiplicand = 21
  MOV R2, #10   @ Multiplier = 10
  MOV R0, #0    @ Result = 0
  MOV R3, #32   @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2       @ Check the LSB of the Multiplier (R2)
  BEQ  no_add       @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  3. …
  4. If the LSB is 1, add the Multiplicand to the Result.
  5. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21  @ Multiplicand = 21
  MOV R2, #10  @ Multiplier = 10
  MOV R0, #0   @ Result = 0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2      @ Check the LSB of the Multiplier (R2)
  BEQ  no_add      @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  4. …
  5. Shift the Multiplicand left by 1 bit (doubling it).
  6. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21  @ Multiplicand = 21
  MOV R2, #10  @ Multiplier = 10
  MOV R0, #0   @ Result = 0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2     @ Check the LSB of the Multiplier (R2)
  BEQ  no_add     @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  5. …
  6. Shift the Multiplier right by 1 bit (so we can check the next bit).
  7. …

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


   MOV R1, #21   @ Multiplicand = 21
   MOV R2, #10   @ Multiplier = 10
   MOV R0, #0    @ Result = 0
   MOV R3, #32   @ Counter = 32 (bits)

mult_loop:
   ANDS R4, R2      @ Check the LSB of the Multiplier (R2)
   BEQ  no_add      @ If it's 0, skip adding
   ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
   LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
   LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
   SUBS R3, R3, #1 @ Decrement Counter
   BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  6. …
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21  @ Multiplicand = 21
  MOV R2, #10  @ Multiplier = 10
  MOV R0, #0   @ Result = 0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2     @ Check the LSB of the Multiplier (R2)
  BEQ  no_add     @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

- Algorithm (for two 32-bit numbers):
  6. …
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result


  MOV R1, #21  @ Multiplicand = 21
  MOV R2, #10  @ Multiplier = 10
  MOV R0, #0   @ Result = 0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2     @ Check the LSB of the Multiplier (R2)
  BEQ  no_add     @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again


@ R0 now contains R1 * R2
```

MUL Multiplication

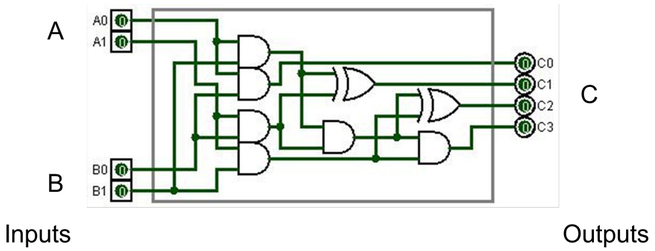
- Why a real **MUL** instruction is better?



2 minutes



MUL Multiplication



- Why a real **MUL** instruction is better:



- The CPU does all this in a **single clock cycle (or a few)** with dedicated, parallel hardware, making it incredibly faster than a software loop.



Key Points: MUL Multiplication on ARM

- **MUL uses repeated shift-and-add algorithm**
  - Initialize Result = 0, loop 32 times for 32-bit numbers
- **Algorithm: check LSB of multiplier, conditionally add**
  - Shift multiplicand left, shift multiplier right each iteration
- **ARM assembly implements MUL loop: LSR, TST, ADD, SUBS**
  - Multiple walkthrough steps show execution flow
- **Hardware MUL: single cycle with parallel circuits**
  - Faster, smaller code, handles edge cases automatically
- **Real MUL better for pipeline: no loop, predictable timing**

PART C

DIV Division on a Simple CPU (ARM)

Division on a simple CPU: how ARM handles DIV in software and hardware.

DIV Division

- Division is even more complex.
- The classic algorithm is shift-and-subtract.
  - It's also called restoring division.

DIV Division

- Algorithm (for 32-bit / 32-bit division):
  1. Initialize a Remainder register to 0.
  2. Initialize a Counter to 32.
  3. Shift the Remainder and Dividend registers together left as one 64-bit value.
     - The top bit of the Dividend shifts into the bottom bit of the Remainder.
  4. Subtract the Divisor from the new Remainder.

DIV Division

- Algorithm (for 32-bit / 32-bit division):
  5. If the result is negative, add the Divisor back (to “restore” it):
     - Put a 0 in the LSB of the Dividend (the **quotient**).
  6. Otherwise, the result is positive:
     - Put a 1 in the LSB of the Dividend (the **quotient**).
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.
  9. After the loop, the Dividend register holds the **Quotient**, and the Remainder register holds the Remainder.

DIV Division

- ARM Assembly Pseudocode (if **DIV** didn't exist) for R0 = R1 / R2 (Quotient) Remainder R3.

- Algorithm (for 32-bit division):
  1. Initialize a Remainder register to 0.
  2. Initialize a Counter to 32.
  3. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32

div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  1. Initialize a Remainder register to 0.
  2. Initialize a Counter to 32.
  3. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32

div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  2. …
  3. Shift the Remainder and Dividend registers together left as one 64-bit value.
     - The top bit of the Dividend shifts into the bottom bit of the Remainder.
  4. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32


div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  2. …
  3. Shift the Remainder and Dividend registers together left as one 64-bit value.
     - The top bit of the Dividend shifts into the bottom bit of the Remainder.
  4. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32


div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  3. …
  4. Subtract the Divisor from the new Remainder.
  5. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32


div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  4. …
  5. If the result is negative, add the Divisor back (to “restore” it):
     - Put a 0 in the LSB of the Dividend.
  6. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32


div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  4. …
  5. If the result is negative, add the Divisor back (to “restore” it):
     - Put a 0 in the LSB of the Dividend.
  6. …

```
@ Input: R1 = Dividend, R2 = Divisor
@ Output: R0 = Quotient, R3 = Remainder
    MOV R0, #0          @ Quotient = 0
    MOV R3, #0          @ Remainder = 0
    MOV R4, #32         @ Counter = 32


div_loop:
    @ Shift R3:R1 left by 1 bit (64-bit shift)
    LSL R3, R3, #1      @ Shift Remainder left
    LSLS R1, R1, #1     @ Shift Dividend left, MSB goes into Carry flag
    ADC R3, R3, #0      @ Add the Carry bit to the Remainder's LSB

    @ Try to subtract divisor from remainder
    SUBS R5, R3, R2     @ R5 = Remainder - Divisor
    BLT  restore        @ If result is negative, we can't subtract

...
```

DIV Division

- Algorithm (for 32-bit division):
  5. …
  6. Otherwise, the result is positive:
     - Put a 1 in the LSB of the **Quotient**.
  7. …

```
...
    BLT   restore       @ If result is negative, we can't subtract

    @ If we get here, subtraction was successful
    MOV R3, R5          @ Update Remainder to the new value (R3 = R3 - R2)
    ORR R0, R0, #1      @ Set the LSB of the Quotient to 1
    B    next_iter


restore:
    @ Subtraction failed, so we do nothing (the LSB of the quotient is already 0 from the initial shift)
    @ We don't need to "add back" because we didn't update R3 yet.


next_iter:
    LSR R0, R0, #1      @ Logical shift right for quotient? Wait, no, this is tricky.
    @ Actually, the quotient is being built in R1 as we shift! The above is a simplification.
    @ A full implementation is more complex to show in a short example.

    SUBS R4, R4, #1     @ Decrement Counter
    BNE div_loop        @ Loop if Counter != 0
```

DIV Division

- Algorithm (for 32-bit division):
  6. …
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.
  9. …

```
...
    BLT   restore       @ If result is negative, we can't subtract

    @ If we get here, subtraction was successful
    MOV R3, R5          @ Update Remainder to the new value (R3 = R3 - R2)
    ORR R0, R0, #1      @ Set the LSB of the Quotient to 1
    B    next_iter


restore:
    @ Subtraction failed, so we do nothing (the LSB of the quotient is already 0 from the initial shift)
    @ We don't need to "add back" because we didn't update R3 yet.


next_iter:
    LSR R0, R0, #1      @ Logical shift right for quotient? Wait, no, this is tricky.
    @ Actually, the quotient is being built in R1 as we shift! The above is a simplification.
    @ A full implementation is more complex to show in a short example.

    SUBS R4, R4, #1     @ Decrement Counter
    BNE div_loop        @ Loop if Counter != 0
```

DIV Division

- Algorithm (for 32-bit division):
  6. …
  7. Decrement the Counter.
  8. If Counter != 0, go back to step 3.
  9. …

```
...
    BLT   restore       @ If result is negative, we can't subtract

    @ If we get here, subtraction was successful
    MOV R3, R5          @ Update Remainder to the new value (R3 = R3 - R2)
    ORR R0, R0, #1      @ Set the LSB of the Quotient to 1
    B    next_iter


restore:
    @ Subtraction failed, so we do nothing (the LSB of the quotient is already 0 from the initial shift)
    @ We don't need to "add back" because we didn't update R3 yet.


next_iter:
    LSR R0, R0, #1      @ Logical shift right for quotient? Wait, no, this is tricky.
    @ Actually, the quotient is being built in R1 as we shift! The above is a simplification.
    @ A full implementation is more complex to show in a short example.

    SUBS R4, R4, #1     @ Decrement Counter
    BNE div_loop        @ Loop if Counter != 0
```

DIV Division

- Algorithm (for 32-bit division):
  8. …
  9. After the loop, the Dividend register holds the **Quotient**, and the Remainder register holds the Remainder.





...

```
@ R0 now contains the Quotient (R1 / R2)
@ R3 now contains the Remainder
```

DIV Division

- This is why the **DIV** instruction is a lifesaver.
- It hides this **incredibly complex** and **slow loop behind a single** mnemonic.

- On CPUs without a hardware divider, the compiler must generate a function call to a library routine that contains code very similar to this loop, which is why division is so slow on those devices.

mnemonic (n) - something such as a very short poem or a special word used to help a person remember something.

ISAs

Conclusion:

Operation	Core Idea	Simple CPU Method	Real ARM Instruction
ADD/SUB	Fundamental logic gate	Single instruction	ADD, SUB
MUL	Repeated Addition	Shift-and-Add Loop	MUL (hardware)
DIV	Repeated Subtraction	Shift-and-Subtract Loop	SDIV, UDIV (hardware)

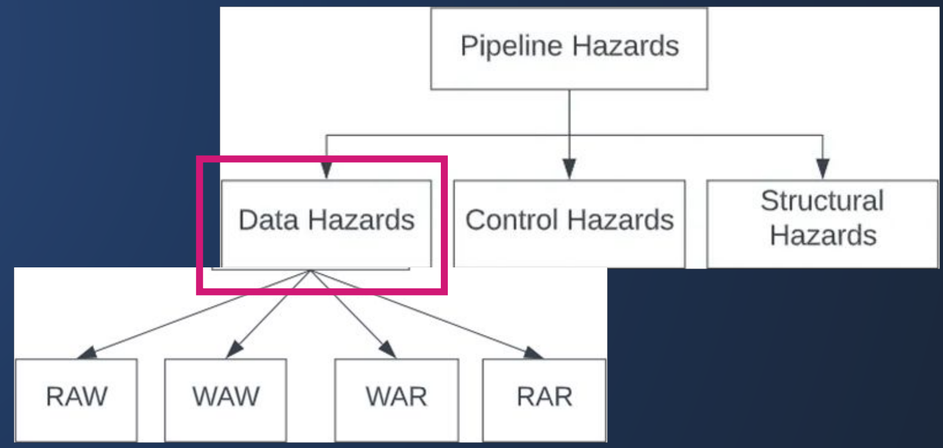


Key Points: DIV Division on ARM

- **DIV uses shift-and-subtract (restoring division) algorithm**
  - More complex than MUL: requires conditional restoration
- **Algorithm: initialize Remainder = 0, loop 32 iterations**
  - Shift left, subtract Divisor, restore if negative
- **Restoring step: if subtraction negative, add Divisor back**
  - Quotient bit = 0 (restore) or 1 (keep subtraction)
- **ARM assembly uses CMP, SUB, ORR, LSL instructions**
  - Code walkthrough demonstrates each loop iteration
- **Hardware DIV hides complex loop behind single mnemonic**
  - ISA trade-off: hardware complexity vs. software simplicity

PART G.1

Pipeline Hazards



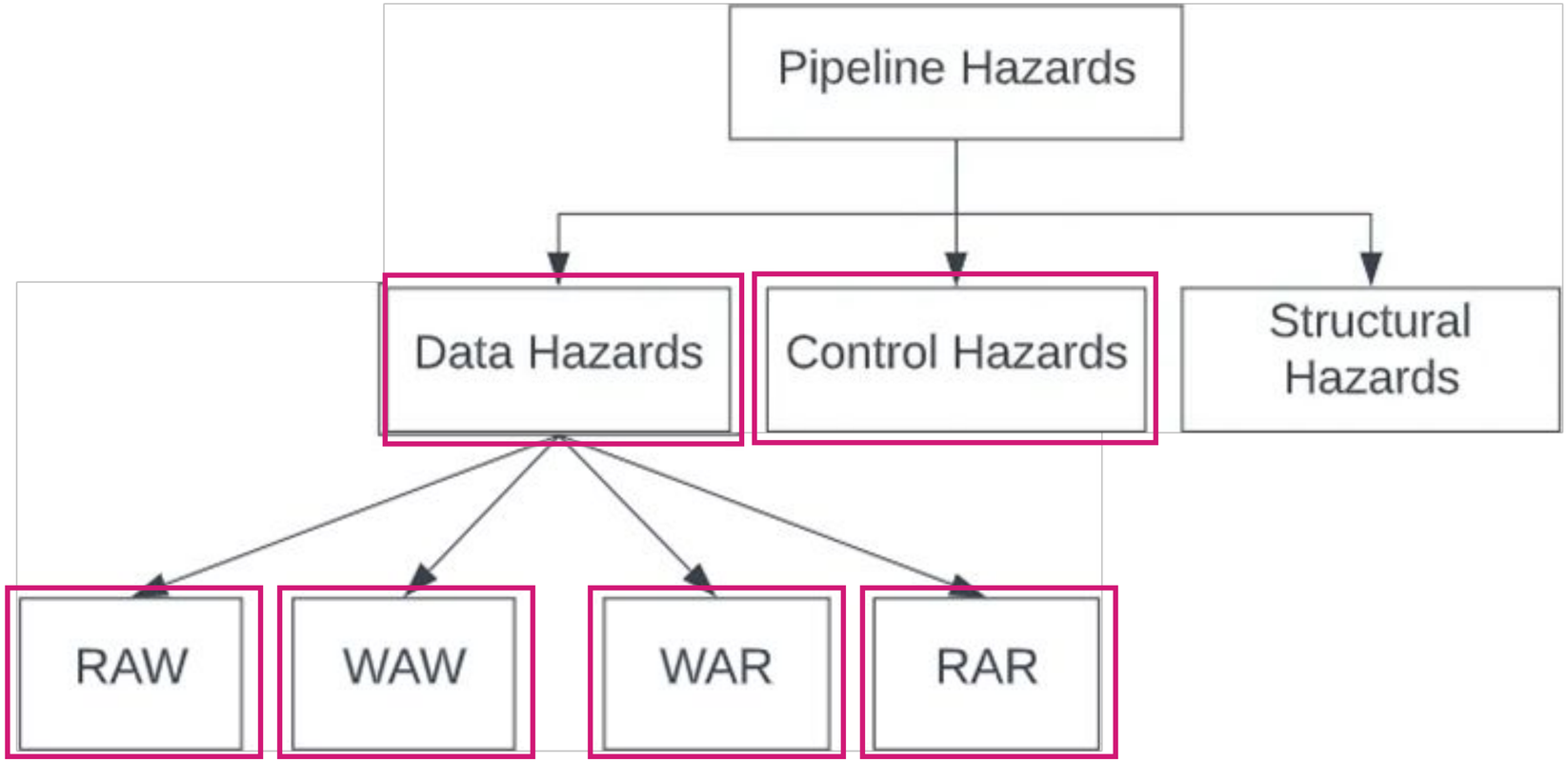
When pipelining goes wrong: data and control hazards, stalls and forwarding.

Pipeline Hazards

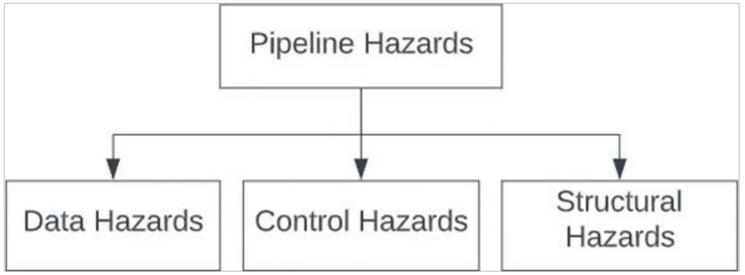
- A **pipeline** must stop for some reason, which is called pipeline hazards.
  - Hazards are problems that occur in the **pipeline**.
  - Rising hazards in the **pipeline** prevent the subsequent instruction from executing during its fixed clock cycle.
  - Hazards reduce the performance from the ideal speedup gained by **pipelining**.

Pipeline Hazards

- There are mainly three types of hazards:



Data Hazards: Overview

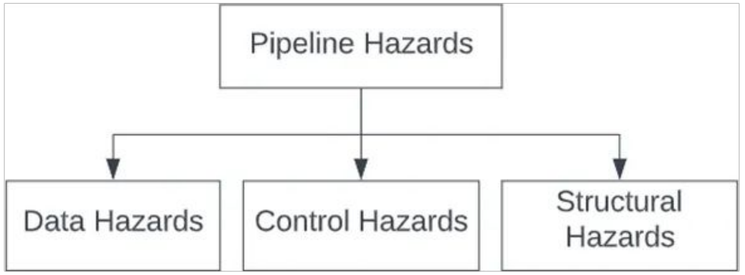


- The **ALU** instruction data hazard is caused by:
  - an ALU instruction writes a new value to a register (e.g., R1), and the register cannot be updated immediately.
- However, as **two separate instructions** can use the same storage; each must be executed in the correct sequence.

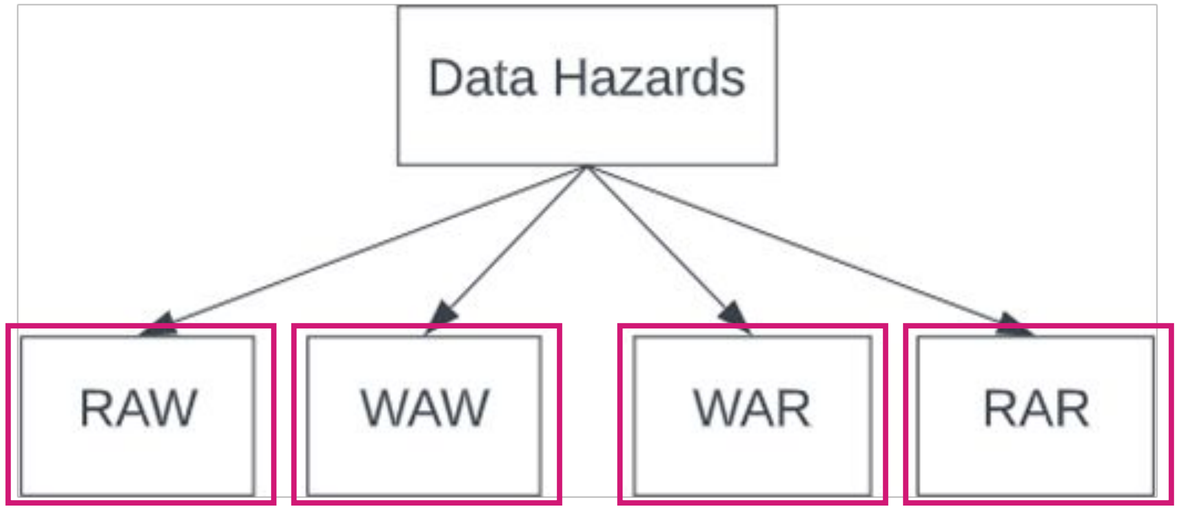
an instruction is dependent on the outcome of a preceding instruction $\wedge$ if the previous instruction's result has not yet been computed $\Rightarrow$ data hazards will occur

source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f

Data Hazards

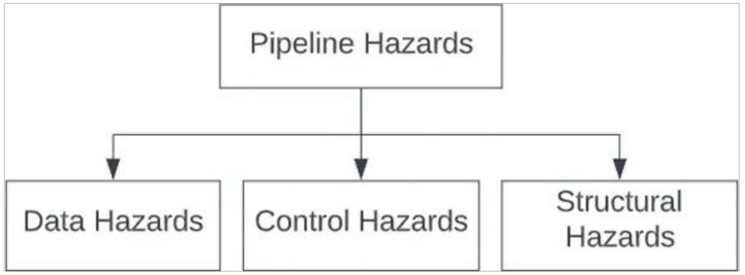


- There are four kinds of Data Hazards:



source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f

Write after Write (WAW)



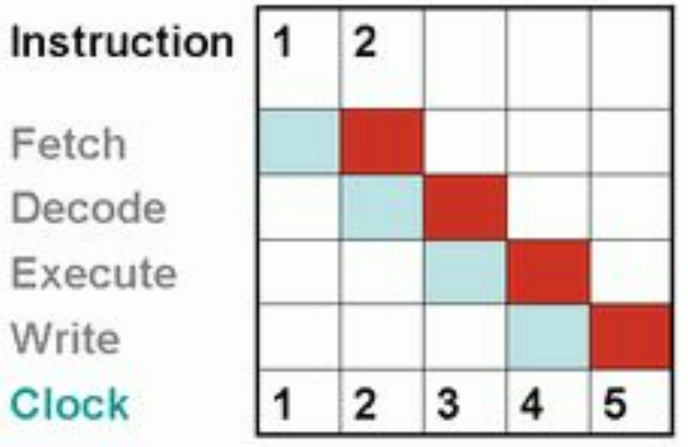
- **Write after Write (WAW)**:

  Output dependency is sometimes known as WAW.

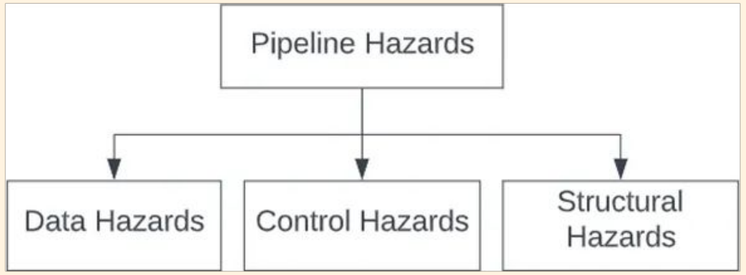
- These hazards arise when an instruction's output register is used for writing after it has been written by a previous instruction.

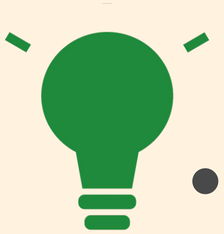
- For example,

  ADD R1, — , — ;
  SUB R1, — , — ;



Write after Write (WAW)





- For example,

ADD R1, R2, R3 ;
SUB R1, R4, R5 ;

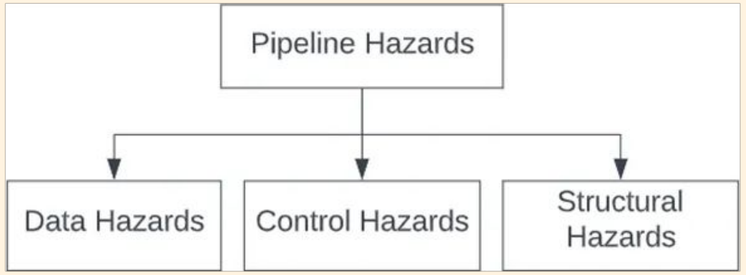
ADD	R1,	R2,	R3	;	Instruction	A:	R1	=	R2	+	R3
SUB	R1,	R4,	R5	;	Instruction	B:	R1	=	R4	-	R5

1

How the Hazard Could Occur:

- Imagine the ADD instruction is delayed.
- For example, the value in register R2 is not yet available because it is the result of a previous, very slow operation (like a load from memory that had a cache miss).
- The **pipeline** sees that the SUB instruction has all of its operands ready (R4 and R5 are available).
- To avoid wasting cycles, the **pipeline** executes the SUB instruction out-of-order, before the stalled ADD.
- The SUB completes and writes its result (R4 - R5) to register R1.

Write after Write (WAW)



- For example,

MUL R0, R1, R2 ;
ADD R0, R3, #1 ;

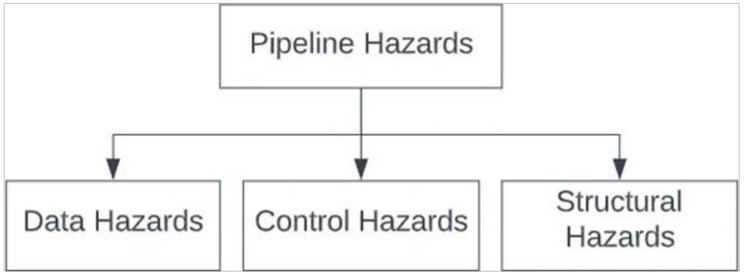
; Assume a pipeline where MUL takes 3 cycles and ADD takes 1 cycle	
MUL R0, R1, R2	; Instruction A (Slow): R0 = R1 * R2
ADD R0, R3, #1	; Instruction B (Fast): R0 = R3 + 1

2

How the Hazard Could Occur:

- The programmer intends for the ADD to execute after the **MUL**, and therefore, the final value in R0 should be the result of the ADD (R3 + 1).
- The result of the **MUL** should be overwritten.

Write after Read (WAR)

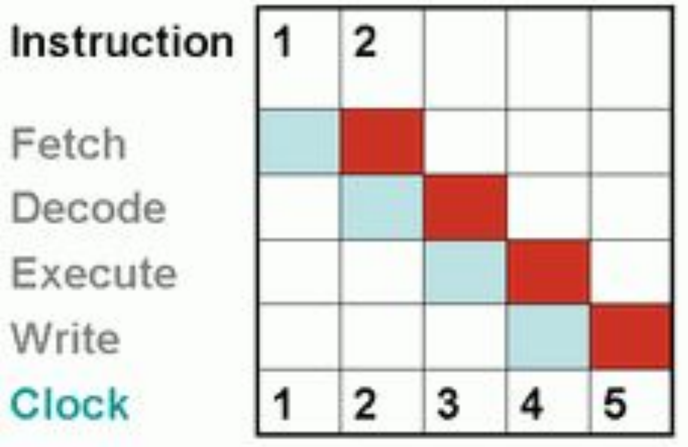


- **Write after Read (WAR)**: Anti-dependency is often known as WAR.

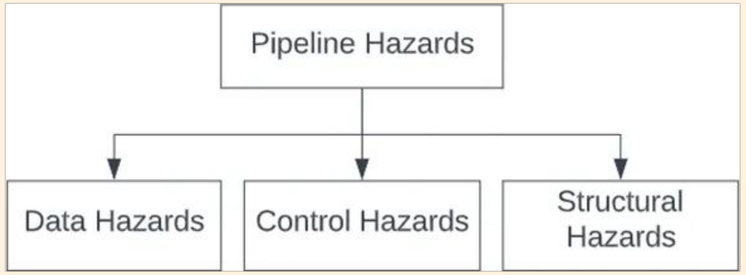
- These hazards arise when an instruction's output register is used after it has been read by a previous instruction.

- For example,

ADD — , R1, — ;
SUB R1, — , — ;



Write after Read (WAR)



- For example,

```
lw  $t0, 0($s0)
add $s1, $s2, $s3
add $t1, $t0, $s4
add $t0, $s5, $s6
```

if it completed before read!!!

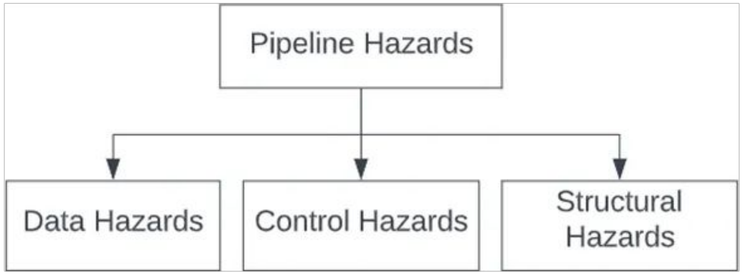
Instruction 1: lw $t0, 0($s0)	**; Load a value into $t0 (Writes in WB)**
Instruction 2: **add $s1, $s2, $s3**	; Does nothing with $t0
Instruction 3: **add $t1, $t0, $s4**	; Reads $t0 (in ID stage) <-- Read
Instruction 4: **add $t0, $s5, $s6**	; Writes $t0 (in WB stage) <-- Write After the Read?

1

How the Hazard Could Occur:

- Cycle n: Instruction 4 is in its WB stage - it writes the new value to register $t0.
- Cycle n: Instruction 3 is in its ID stage - it needs to read the value of $t0.
  - If the register file is written in the first half of the cycle and the read happens in the second half, Instruction 3 might read the new value written by Instruction 4, instead of the old value it was supposed to read.

Read after Read (RAR)

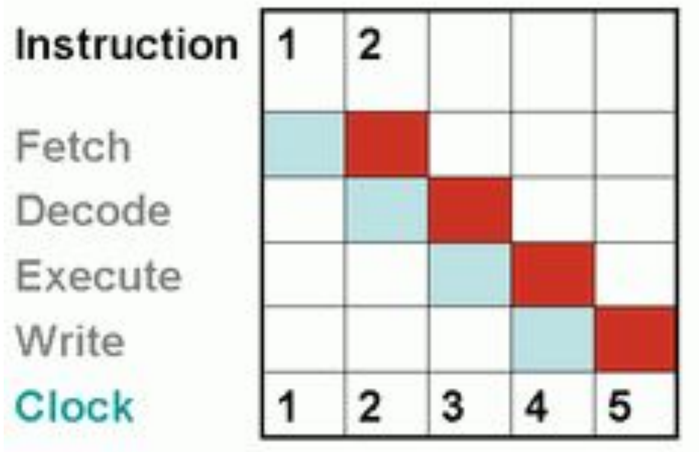


- **Read after Read (RAR)**:

  When two instructions read from the same register, RAR occurs.

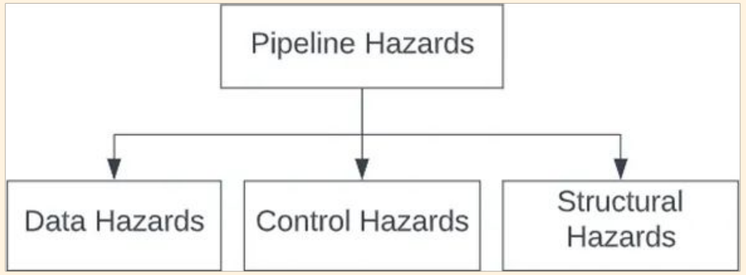
- For example,

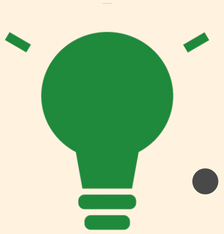
  ADD — , R1, — ;
  SUB — , R1, — ;



- These Read after Read (RAR) hazards ***may*** **present an issue for the processor** , reading a register value *usually* does not modify the register value, but can cause a **stall**.

Write after Read (WAR)





- For example,

```
ADD R4, R5, R1 ;
SUB R6, R7, R1 ;
CMP R8, R1     ;
```

Insufficient number of read ports!!!

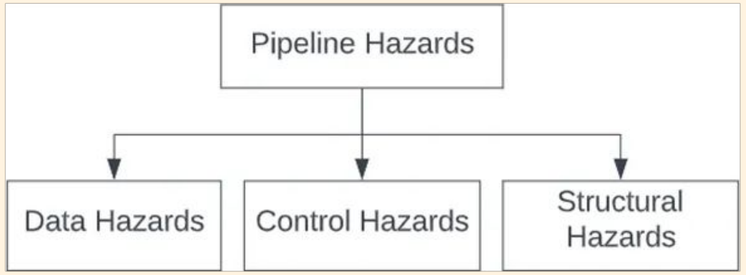
@ R1 is used as a common source register					
ADD	R4,	R5,	R1	@ Instruction A: Reads R1	
SUB	R6,	R7,	R1	@ Instruction B: Reads R1	
CMP	R8,	R1		@ Instruction C: Reads R1 (and sets flags)	

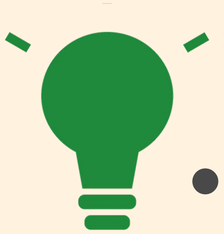
How/When Does the Hazard Occur:

- Imagine a register that, when read, is automatically reset to zero.
- This is sometimes a behavior of hardware flags or special-purpose registers in very old systems, but never for general-purpose registers like $s0.
- A more realistic scenario is a structural hazard.
- **A simple register file might have only one read port.**
- This means it can only supply the value of one register per clock cycle.

1

Write after Read (WAR)





- For example,

```
add $t0, $s1, $s0
sub $t1, $s2, $s0
and $t2, $s3, $s0
```

# $s0 is used as a common constant value (e.g., 10)					
add $t0,	$s1,	$s0		# Instruction	A: Reads $s0
sub $t1,	$s2,	$s0		# Instruction	B: Reads $s0
and $t2,	$s3,	$s0		# Instruction	C: Reads $s0

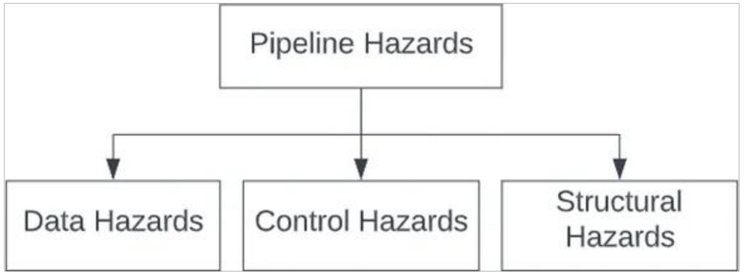
Not allowed to read at the same time!!!

How/When Does the Hazard Occur:

2

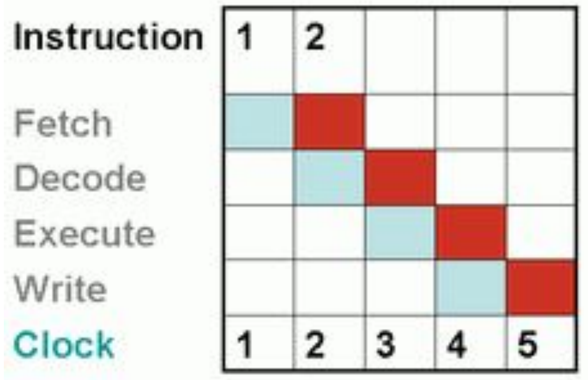
- **Cycle 2** (ID stage for sub): The sub instruction enters the ID stage.
  - The register file reads $s2. It cannot read $s0 yet.
  - The sub instruction must **stall** (**hazard occurs if no stall mechanism**).
- **Cycle 3** (ID stage for sub continued): The register file reads $s0 for the sub instruction.

Read after Write (RAW)



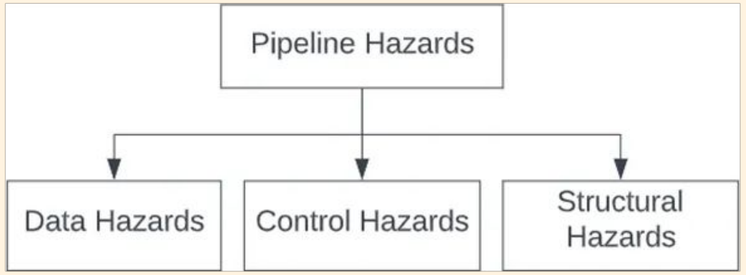
- **Read after Write (RAW)**: True dependency or Flow dependency are other terms for RAW.
- It happens when a value generated by one instruction is required by another instruction.
- For example,
  - These hazards require the **use of stalls**.

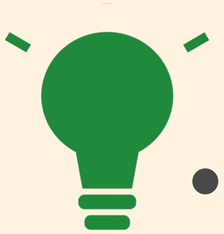
```
ADD R1, — , — ;
SUB — , R1, — ;
```



  - A **pipeline** stall is a delay in the execution of an instruction to resolve a hazard.)

Read after Write (RAW)





- For example,

add $s0, $t1, $t2
sub $t3, **$s0**, $t4

add $s0,	$t1, $t2		# Instruction 1: Writes to $s0
sub $t3,	**$s0,**	**$t4**	# Instruction 2: Reads **$s0**

ID before WB!!!

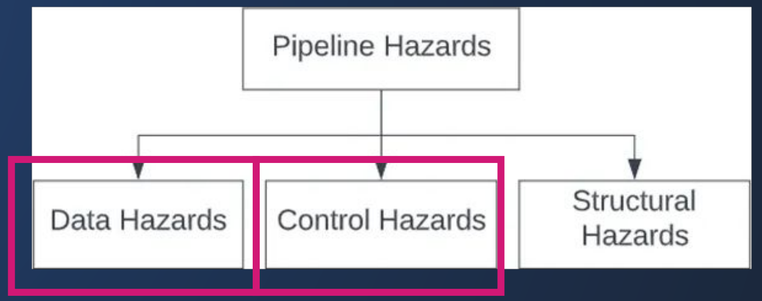
1

What was the "RAR Hazard"?:

- Instruction 1 (add) writes its result to $s0 in its WB (Write-Back) stage.
- Instruction 2 (sub) reads its source registers ($s0, $t4) in its ID (Instruction Decode) stage.
- In the **pipeline**, the ID stage of sub occurs before the WB stage of add. This means sub tries to read the old value of $s0 before add has had a chance to update it.
- Without intervention, the sub would use incorrect data.

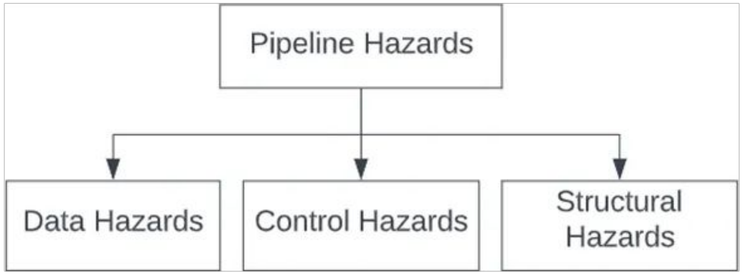
PART G.2

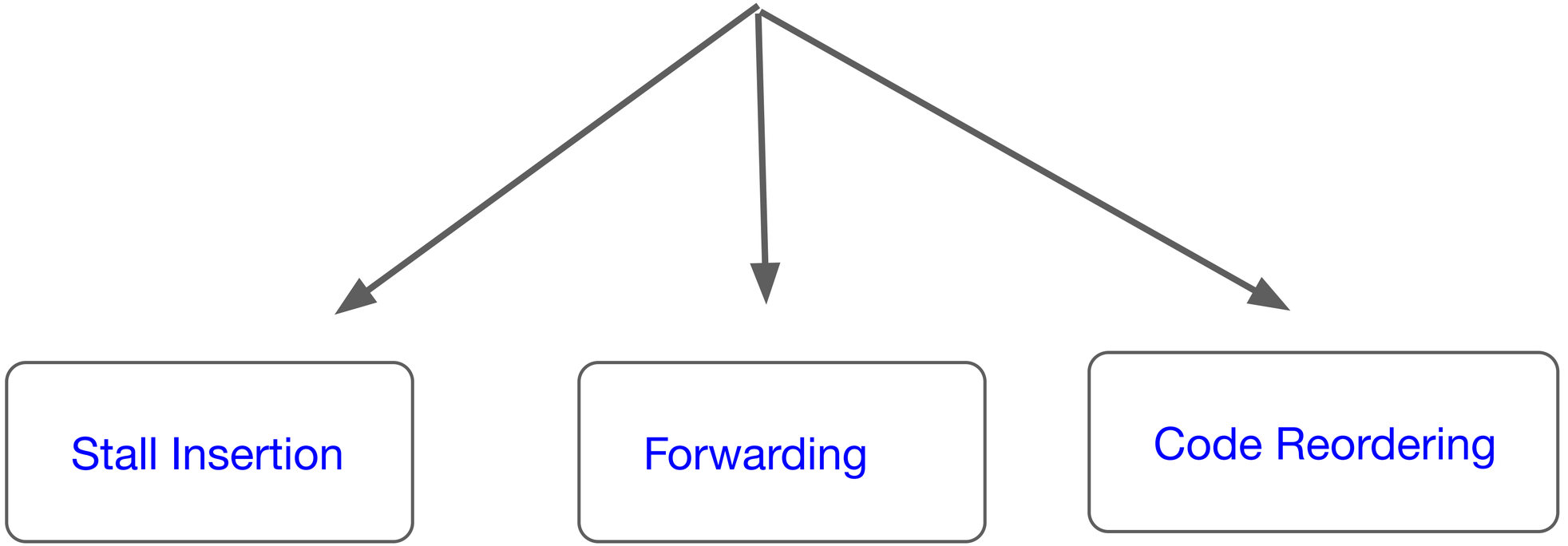
Pipeline Hazards: Handling



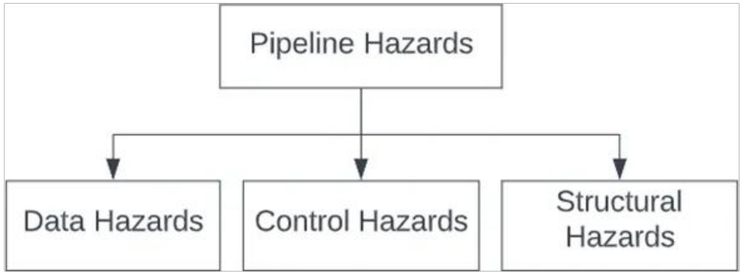
When pipelining goes wrong: data and control hazards, stalls and forwarding.

Data Hazards: Handling





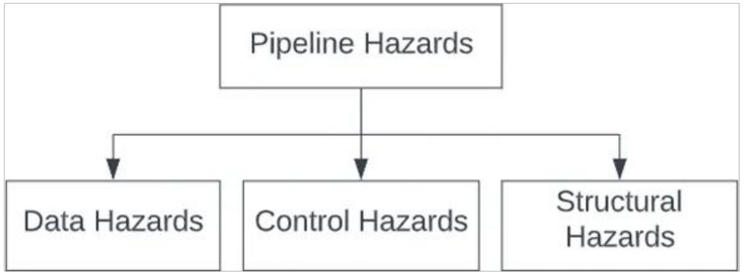
Data Hazards: Handling



Stall Insertion

1. Stall Insertion:
   - This method inserts one or more stalls (no-op instructions: “bubble”) into the **pipeline**.
   - **Delays execution** of the current instruction until the required operand is written to the **register file**.
   - This method **reduces pipeline efficiency** and **throughput**.

Data Hazards: Handling



Forwarding

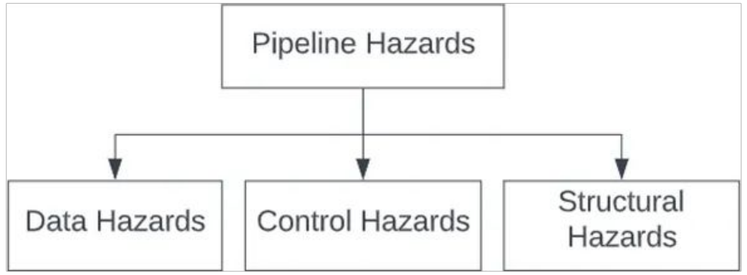
Forwarding:

It modifies the **pipeline** by adding special circuitry.

- This method works because the required values take less time to go via a circuit (wire) than a **pipeline segment** does to compute its result.

2.

Data Hazards: Handling

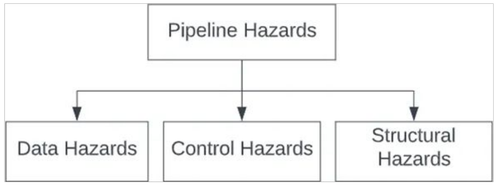


Forwarding

Code Reordering:

- A **hardware-dependent compiler** (specialized software) is required to reorder code instructions.

Stall Insertion



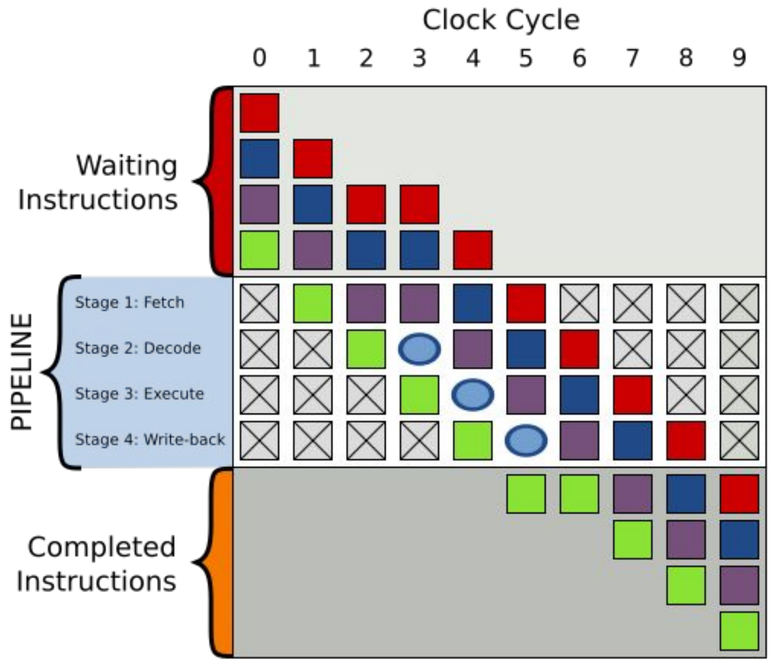
- **Read after Write (RAW)**:

  True dependency or Flow dependency are other terms for RAW.

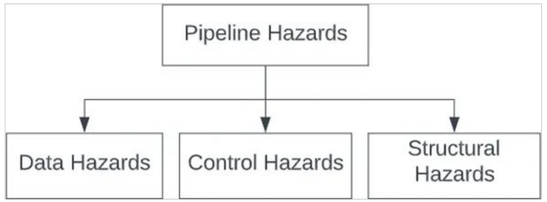
- It happens when a value generated by one instruction is required by another instruction.

  ADD R1, — , — ;

  SUB — , R1, — ;



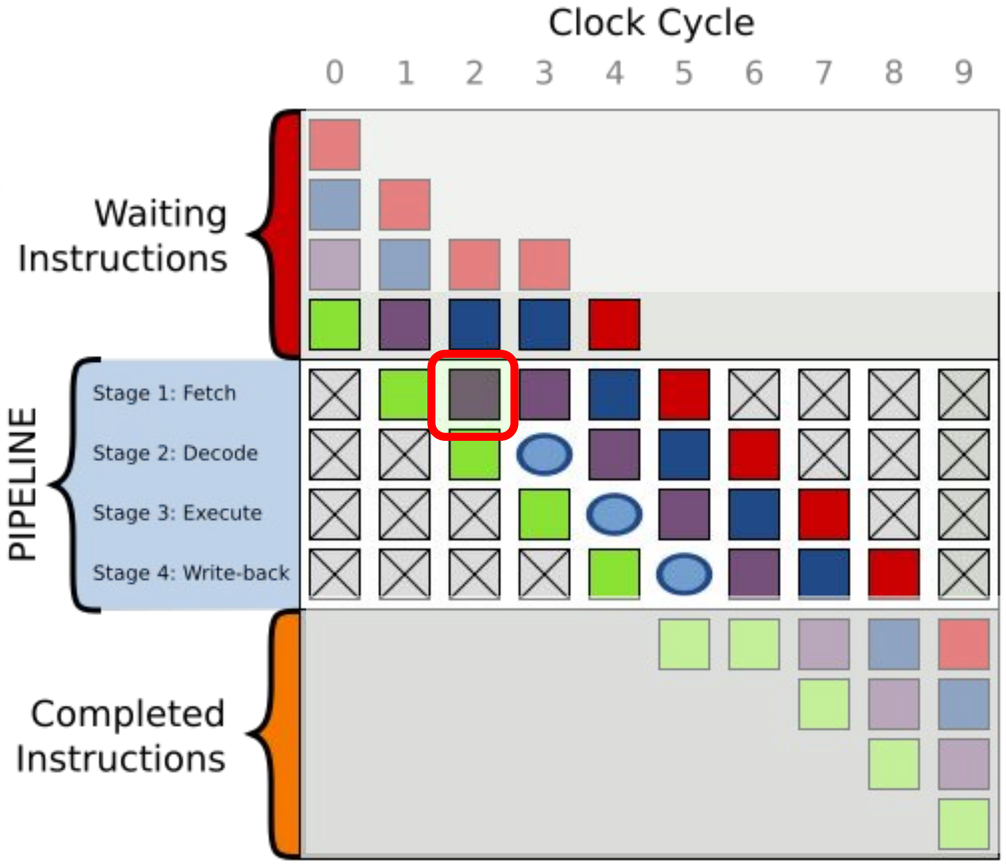
Stall Insertion: Example



- A **delay in fetching** (stall) the **purple instruction in cycle #2** leads to a **bubble** being created.

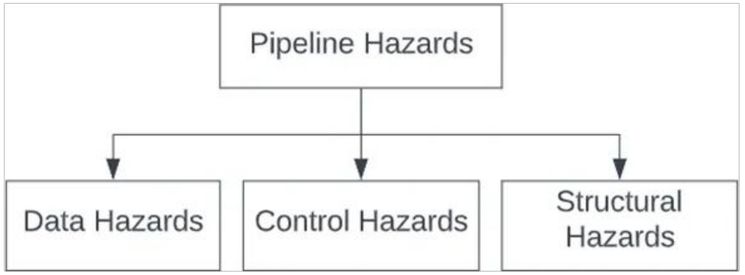


- All instructions after it are delayed as well.

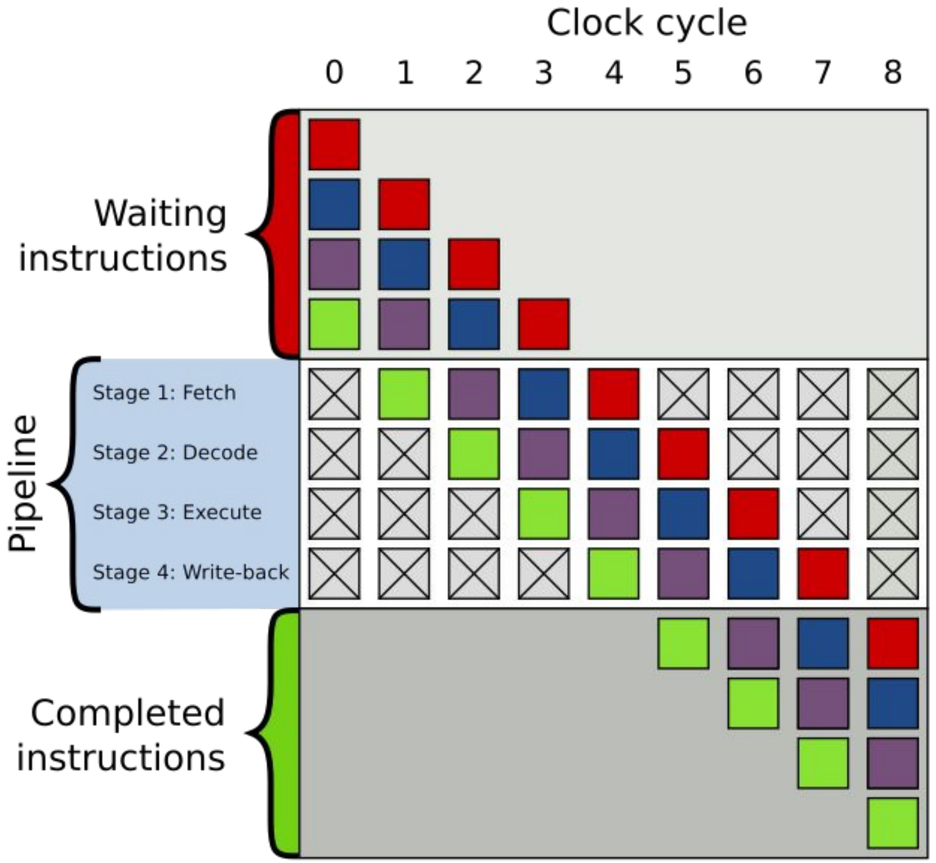


source: https://en.wikipedia.org/wiki/Pipeline_stall

Stall Insertion



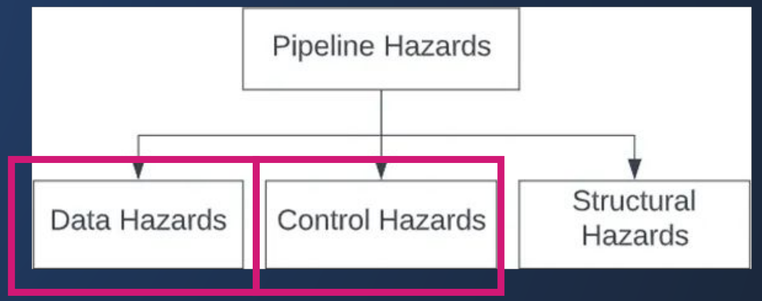
- No stall; same example.
- Without a delay in fetching the purple instruction in cycle #2.
- *No efficiency reduction*



source: https://en.wikipedia.org/wiki/Pipeline_stall

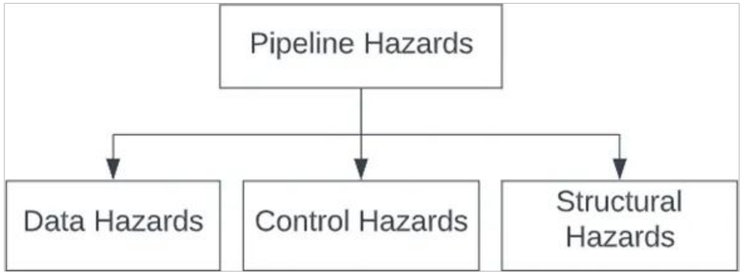
PART G.2

Pipeline Hazards: Handling
More advanced cases

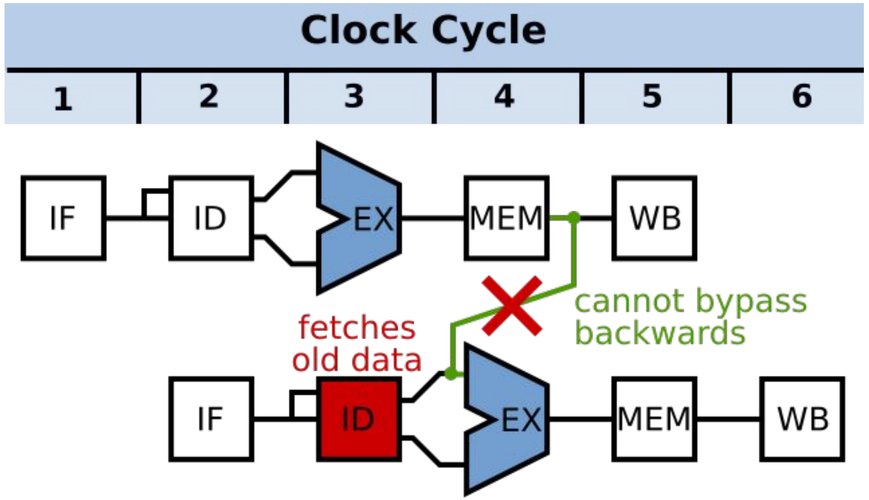


When pipelining goes wrong: data and control hazards, stalls and forwarding.

Classic RISC Pipeline



- A classic RISC **pipeline**, with five stages:
  - IF = **Instruction Fetch**
  - ID = **Instruction Decode**
  - EX = Execute
  - MEM = **Memory access**
  - WB = Register **write back**



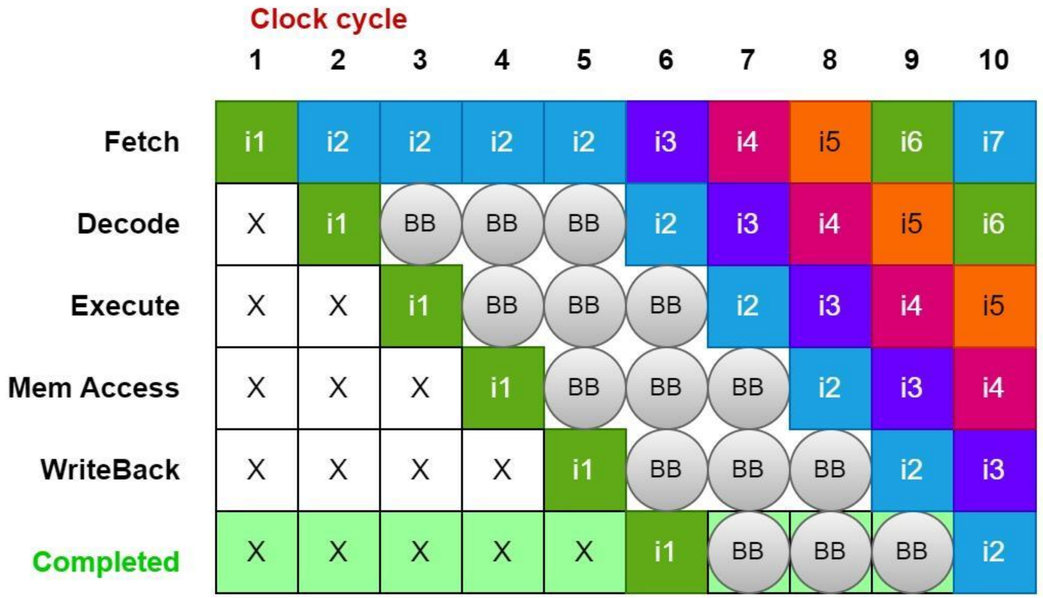
source: https://en.wikipedia.org/wiki/Pipeline_stall

- In this example,
  - data available after the MEM stage (4th stage) of the first instruction is **required** as input by the EX stage (3rd stage) of the second instruction.

Stall Insertion

- The stalls occur in the 5-stage **pipeline**; the pipeline may be forced to stall until the instruction can be successfully executed.

- We ideally expect our CPU to work at full throughput, i.e., CPI = 1.
- But when the **pipeline** stalls, it reduces the throughput/performance of the CPU because CPI increases.



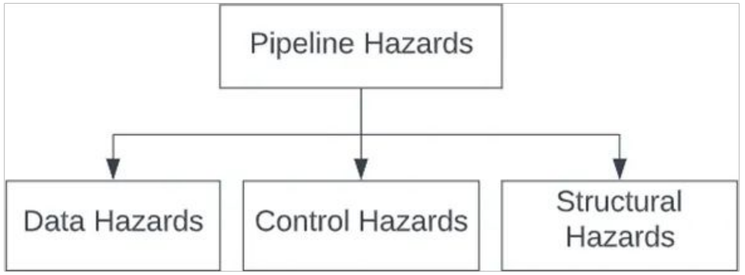
- For a non-ideal CPU:

$$\text{Clock cycles Per Instruction} = 1 + \text{Stall cycles per instruction.}$$

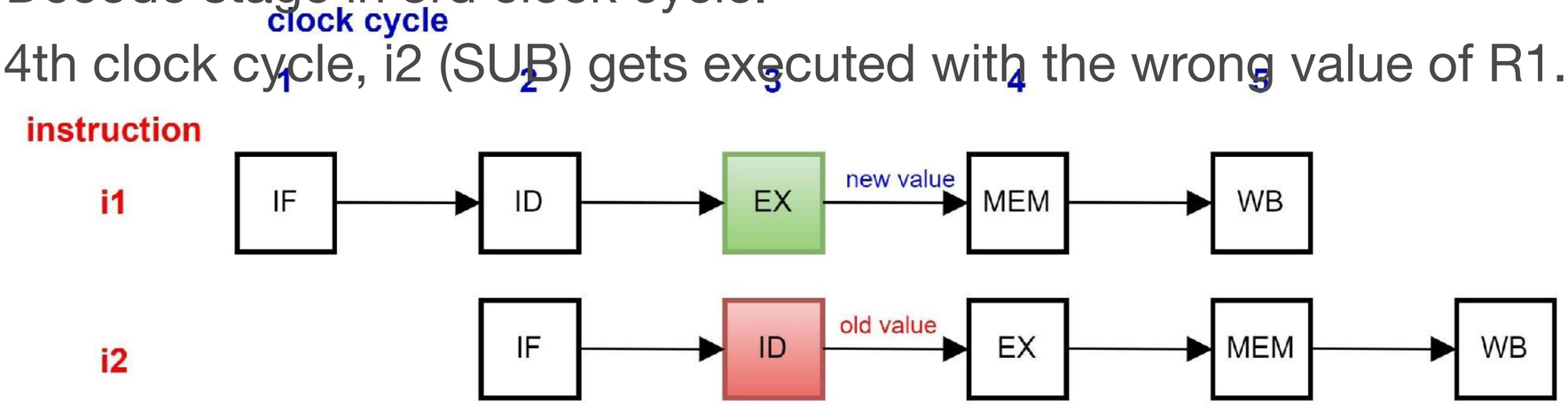
$$\implies \text{CPI}_{\text{non-ideal}} = \text{CPI}_{\text{ideal}} + \text{Stall cycles per instruction}$$

source: https://chipmunklogic.com/digital-logic-design/designing-pequeno-risc-v-cpu-from-scratch-part-3-dealing-with-pipeline-hazards/

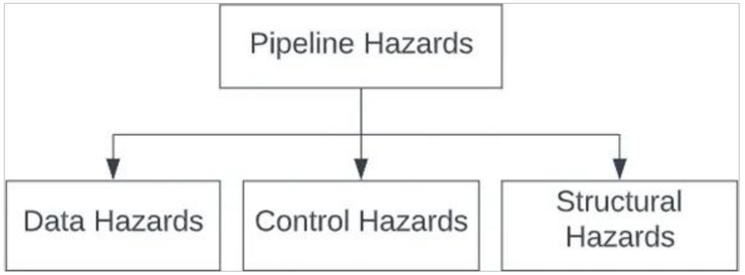
Data/Operand Forwarding



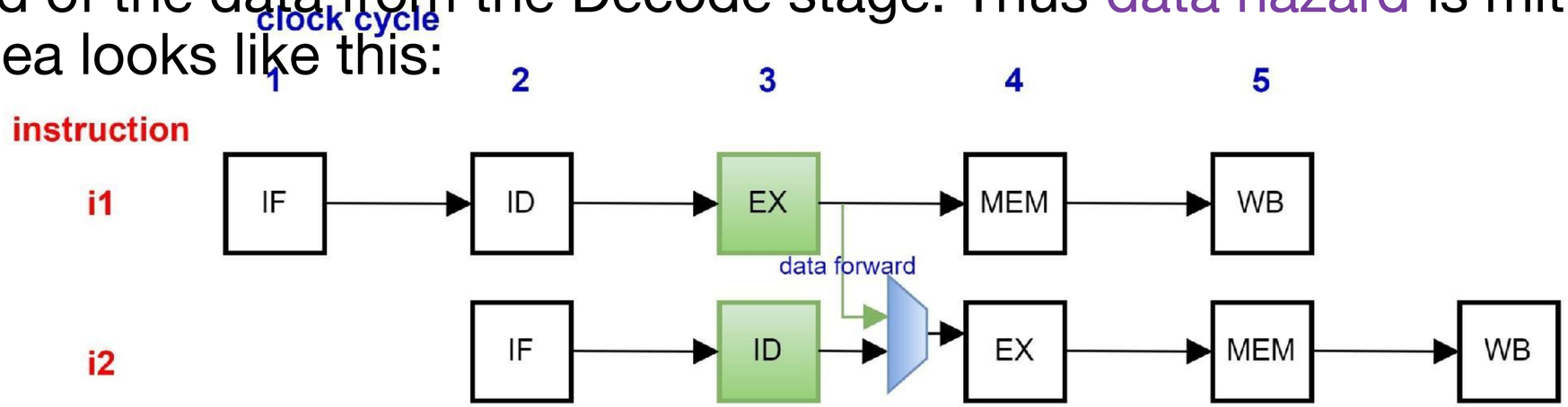
- Assume two adjacent instructions in a 5-stage **pipeline**, i1 (ADD) and i2 (SUB), have a RAW data dependency between them as both are accessing a register R1.
- The CPU should stall the instruction i2 (SUB) until i1 (ADD) writes back the result to the register R1.
- If the CPU has no stalling mechanism, the older value is read by i2 (SUB) from R1 at the Decode stage in 3rd clock cycle.
- In the 4th clock cycle, i2 (SUB) gets executed with the wrong value of R1.



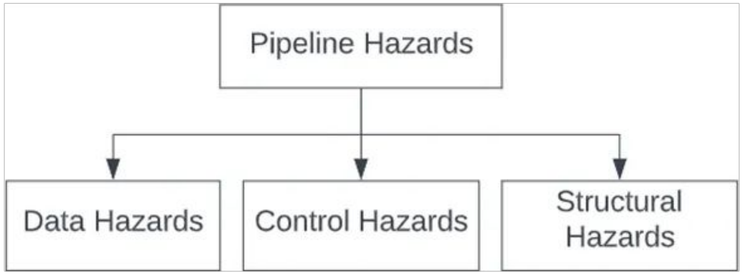
Data/Operand Forwarding



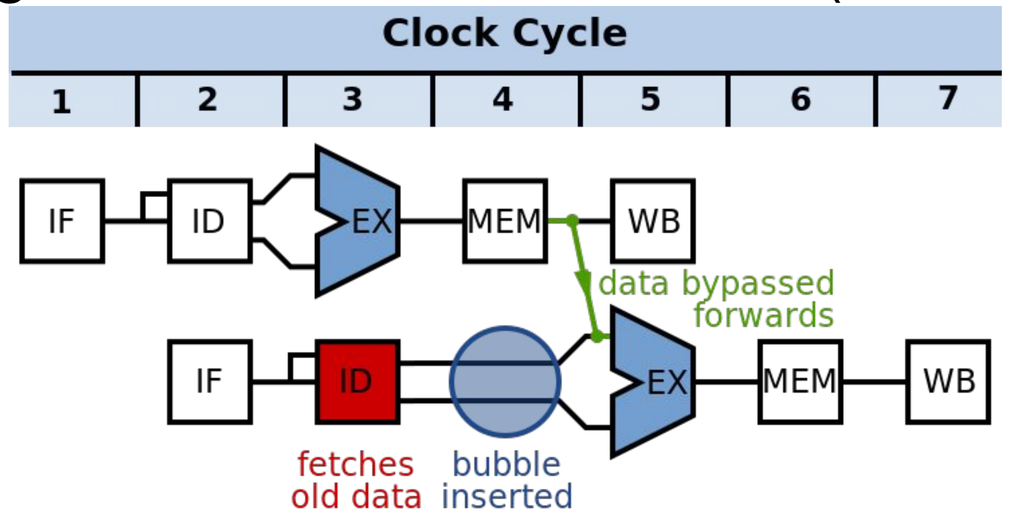
- If we look closely at the **pipeline**, we already have the result of i1 available in the 3rd clock cycle.
- It is not written back to the **register file**, but still the result is available at the output of the Execute stage.
- If we can detect the data dependency and then “forward” this data to the Execute stage input, then the next instruction can use the forwarded data instead of the data from the Decode stage. Thus data hazard is mitigated! The idea looks like this:



Classic RISC Pipeline

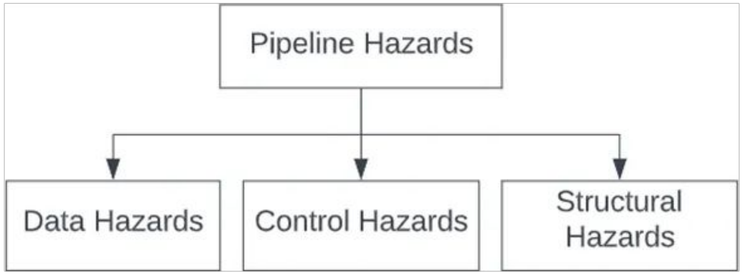


- The example shows a bubble being inserted into a classic RISC **pipeline**.
- At the 4th stage, it cannot do forwarding.
- Without a bubble, the EX stage (3rd stage) only has access to the output of the previous EX stage. Thus, adding a bubble resolves the time dependence without needing to propagate data backwards in time (which is impossible).



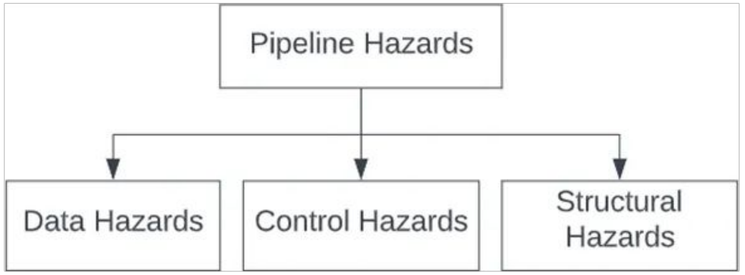
source: https://en.wikipedia.org/wiki/Pipeline_stall

Control Hazards



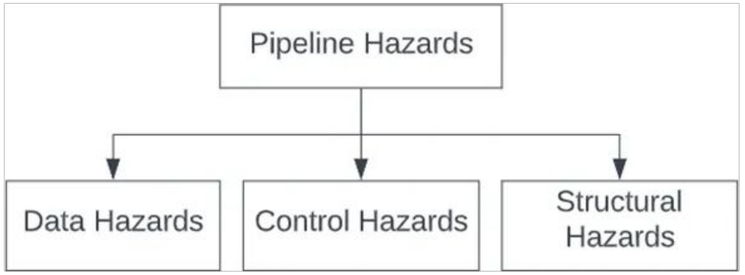
- Control Hazards are caused by Jump/Branch instructions.
- Jump/Branch instructions are the flow control instructions in the ISA of the CPU.
- When the control reaches a Jump/Branch instruction, the CPU has to make a decision whether to take the branch or not.
- One of the following actions should be taken.
  - Fetch the next instruction at PC+4 (branch not taken), or
  - Fetch the instruction at the branch target address (branch taken).

Control Hazards



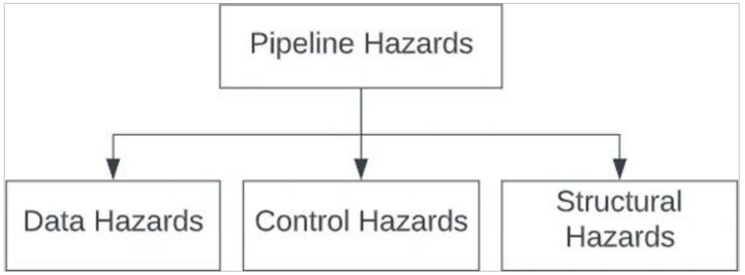
- Whether the decision was right or wrong is figured out only at the Execute stage, where the result of the branch instruction is computed.
- Depending on whether the branch is taken or not, the branch address (the address to which the CPU should branch out) is resolved.
- If the decision made earlier was wrong, all the instructions in the **pipeline** fetched and decoded until that clock cycle should be discarded.
- Because those instructions should not be executed at all.

Control Hazards



- This is done by flushing the **pipeline** and fetching the instruction at the branch address in the next clock cycle.
- Flushing invalidates the instructions and converts them into NOPs or bubbles.
  - This incurs a significant amount of clock cycles as a penalty.
- This is called a branch penalty.
- Control hazards thus have the worst effect on the performance of the CPU.

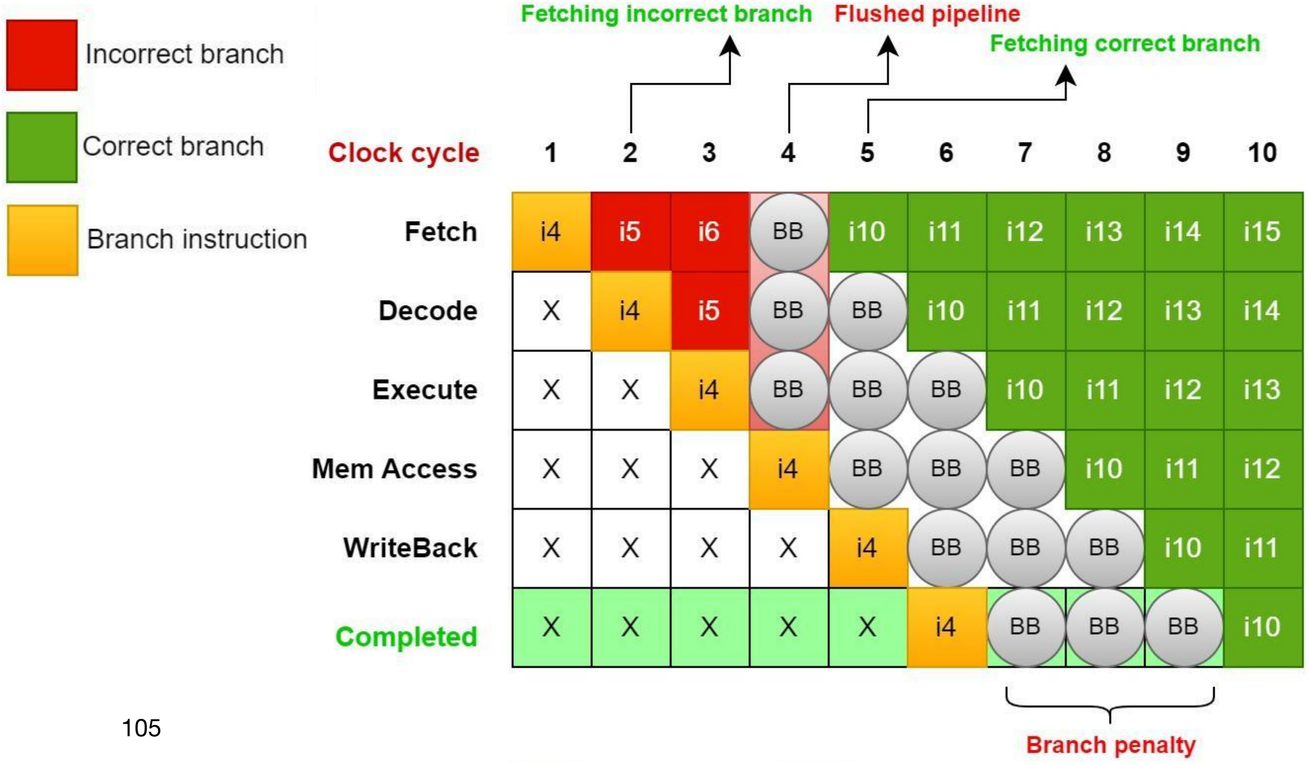
Flushing



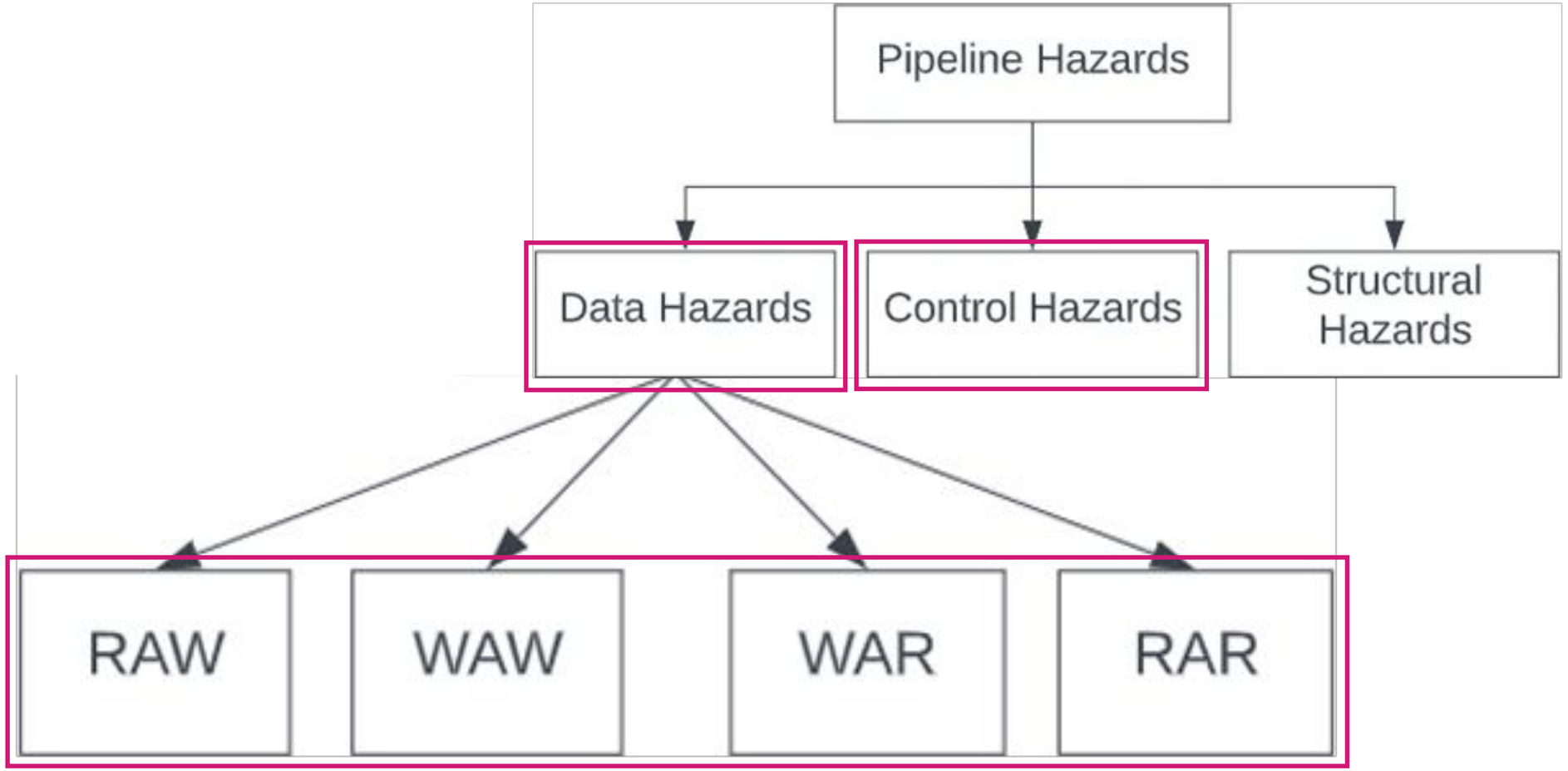
- Example, i10 completes the execution in 10th clock cycle, but it should have completed the execution in 7th clock cycle.
- A penalty of three clock cycles was incurred because the wrong branch (i5) was taken.

- Flush had to be done in the **pipeline** when this was identified by the Execute stage in the 4th clock cycle.
- How will this impact the CPU performance?
- If CPU has 30% branch instructions, CPI will become:
  - 50% performance down.

$$\text{CPI} = (\text{CPI}_{ideal} \times 0.70) + (3 \times 0.30) \approx 2$$



Pipeline Hazards



source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f



Key Points: Data Hazards Handling

- **Stall insertion: add NOP/bubble to delay dependent instructions**
  - Reduces pipeline efficiency; CPI increases above ideal 1.0
- **Data forwarding: bypass register file, route result directly**
  - Special circuitry detects dependency at EX stage output
- **Forwarding limitation: some cases still need a stall bubble**
  - EX output available before WB completes
- **Control hazards: caused by branch/jump instructions**
  - Wrong branch decision discovered only at Execute stage
- **Flushing: invalidate wrong-path instructions as NOPs**
  - Significant clock cycle penalty (e.g., 3 cycles wasted)

PART G

Lab 1 Worksheet:
for MIPS Instruction Set Arch

Hands-on MIPS assembly: registers, R-type instructions and arithmetic.

Objectives

- Understand what instructions look like in machine code.
- Learning Assembly programming
  - Learn about the MIPS Assembly language
  - Write simple MIPS programs
  - Use system calls for simple input and output
    - using https://tio.run/#
    - using **MARS.jar** *(MIPS Assembly Simulator) - download.*

MIPS

- MIPS stands for Microprocessor without Interlocked **Pipeline** Stages
- Full name: Microprocessor without Interlocked Pipeline Stages
- Origin: Developed by MIPS Computer Systems in the 1980s.
- Instruction format:
  - Fixed 32-bit instructions,
  - Simple **3-operand** style (**ADD** R1, R2, R3).
- Use in education:
  - Extremely popular in teaching computer architecture because it's **clean** and **easy** to read.
  - Tools like SPIM, MARS simulate it.

MIPS Registers:

- 32 general-purpose registers.

Result value registers:
- Named **$v0, $v1**

Argument registers:
- Named **$a0 – $a3**

Temporary registers:
- Named **$t0, $t1 — $t9**

… and a few more.

Variable registers:
- Named **$s0, $s1 — $s7**

#	Name	Notes
0	zero	always contains 0
1	at	reserved for assembler
2	v0	expression evaluation and function results
3	v1	
4	a0	arguments passed to procedures and functions
5	a1	
6	a2	
7	a3	
8	t0	temporary storage – can be overwritten by called routines
…	…	
15	t7	

#	Name	Notes
16	s0	variable storage – must be saved by called routines
…	…	
23	s7	
24	t8	more temporary storage
25	t9	
26	k0	reserved for OS use
27	k1	
28	gp	global area pointer
29	sp	stack pointer
30	fp	frame pointer
31	ra	return address register

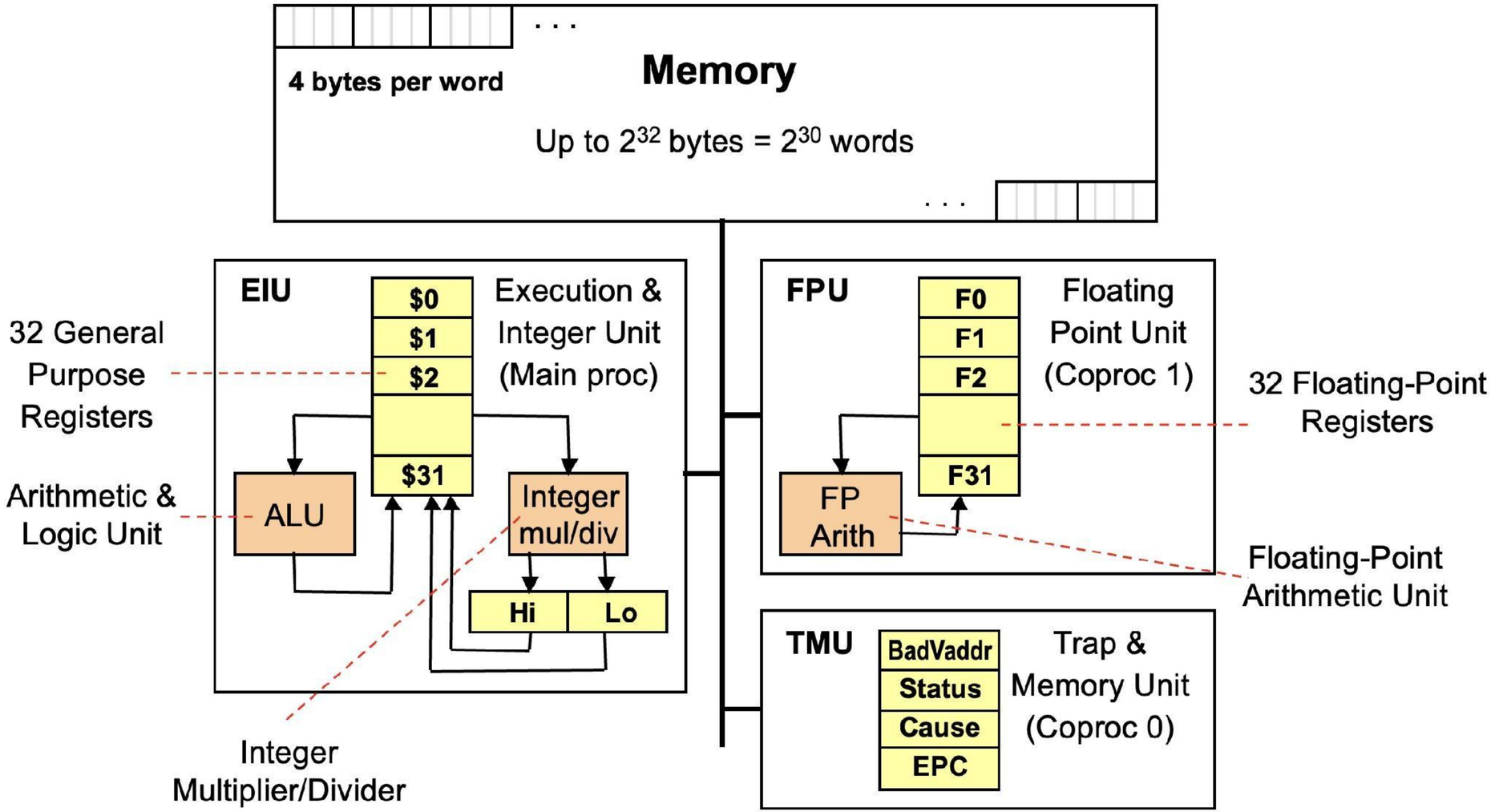
Source: https://www.cs.rit.edu/~paw/250/notes/135e/,notes.221/02/MIPS%20registers.pdf

We focus on MIPS in this subject, so you will meet all of these registers.

Outline

- Overview of the MIPS Architecture
- R-Type Instruction Format
- R-type Arithmetic, Logical, and Shift Instructions
- I-Type Instruction Format and Immediate Constants
- I-type Arithmetic and Logical Instructions
- Pseudo Instructions
- Multiplication and Division Instructions

Overview of the MIPS Architecture

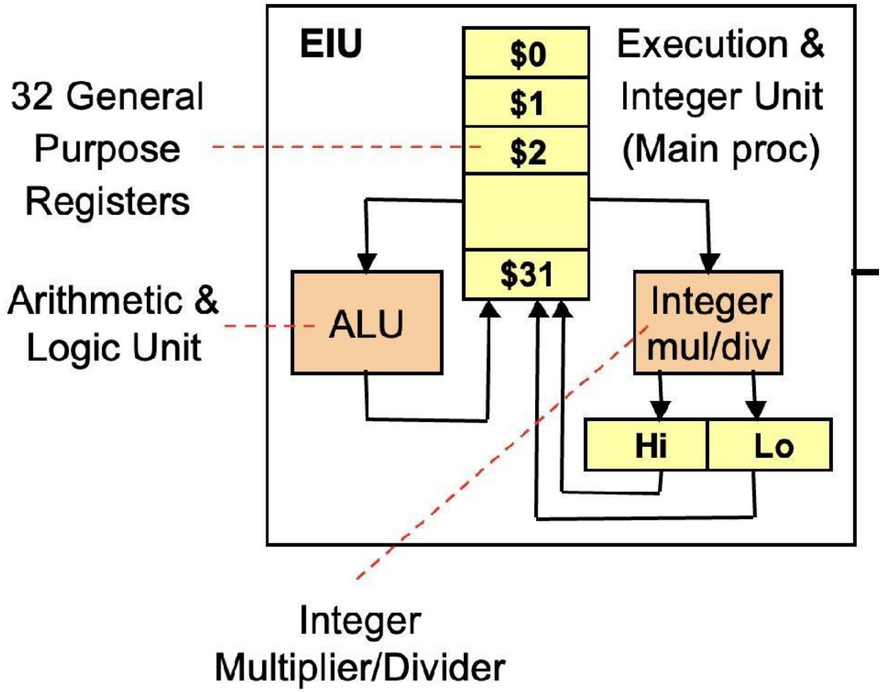


Overview of the MIPS Architecture

Execution & Integer Unit (EIU)

The core block responsible for all integer-based operations:

- **32 General-Purpose Registers:** Fast 32-bit storage ($0 to $31) where register $0 is hardware-wired to always contain the value 0.
- **Arithmetic & Logic Unit (ALU):** Performs primary mathematical (addition, subtraction) and logical evaluations.
- **Integer Multiplier/Divider:** A dedicated hardware unit that processes multiplication and division, outputting results into specialized **Hi** and **Lo** registers.

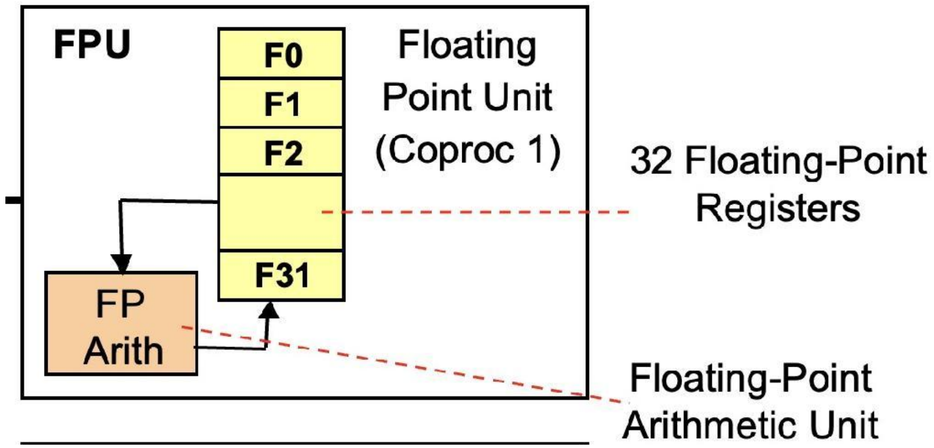


Overview of the MIPS Architecture

Floating-Point Unit (FPU)

A dedicated coprocessor (Coprocessor 1) designed for high-performance floating-point math:

- **32 Floating-Point Registers (F0–F31):** Dedicated 32-bit registers used to store single-precision and double-precision real numbers.
- **Floating-Point Arithmetic Unit:** An independent execution unit that performs arithmetic operations on floating-point operands in parallel with the main CPU.



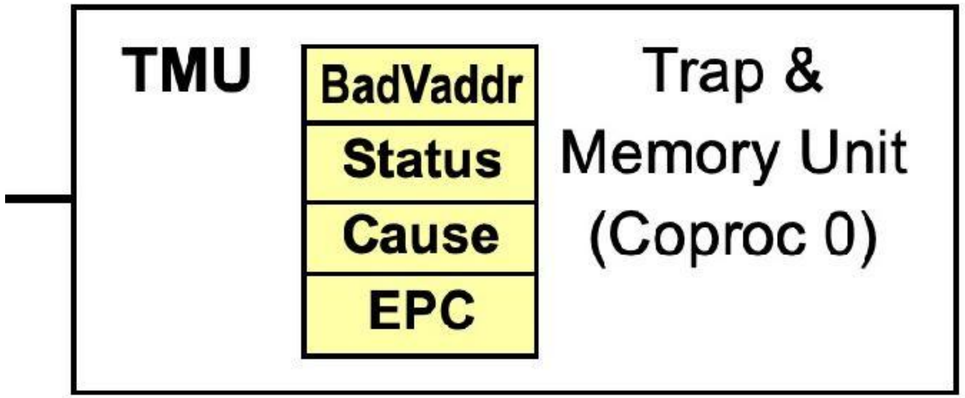
(More on Floating Points in future weeks.)

Overview of the MIPS Architecture

Trap & Memory Unit (TMU)

Coprocessor 0 (Coproc 0) manages system control, exceptions, and virtual memory:

- **System Control & Exceptions:**
  - Serves as Coprocessor 0 to manage **hardware exceptions**, critical for handling **Pipeline Data Hazards**.
    - EPC (Exception Program Counter).
- **Status & Cause Storage:**
  - Stores detailed execution states, including **Carry**, **Borrow**, and **Overflow** **cause** bits.



(More on Carry, Borrow, Overflows in future weeks.)

MIPS General-Purpose Registers

- 32 General Purpose Registers (GPRs)
  - All registers are 32-bit wide in the MIPS 32-bit architecture
  - Software defines names for registers to standardize their use
  - Assembler can refer to registers by name or by number ($ notation)

Name	Register	Usage
$zero	$0	Always 0 (forced by hardware)
$at	$1	Reserved for assembler use
$v0 – $v1	$2 – $3	Result values of a function
$a0 – $a3	$4 – $7	Arguments of a function
$t0 – $t7	$8 – $15	Temporary Values
$s0 – $s7	$16 – $23	Saved registers (preserved across call)
$t8 – $t9	$24 – $25	More temporaries
$k0 – $k1	$26 – $27	Reserved for OS kernel
$gp	$28	Global pointer (points to global data)
$sp	$29	Stack pointer (points to top of stack)
$fp	$30	Frame pointer (points to stack frame)
$ra	$31	Return address (used for function call)

MIPS CPU Register Controls

Program Counter (PC)

Tracks which code instruction to run next by storing the active memory address. (e.g. +4 byte address)

Stack Pointer (SP)

Tracks the constantly changing top of the active working memory stack.

Frame Pointer (FP)

Acts as a fixed reference point for the current function's local variables and arguments.

		Instructions	Memory	Memory
	Feature	**Program Counter (PC)**	**Stack Pointer (SP)**	**Frame Pointer (FP / BP)**
	Primary Role	Tracks code execution.	Tracks the top of the call stack.	Tracks the base of the current function's stack frame.
	Points to	The **memory address of the next instruction.**	The most recently pushed data or the next free slot on the stack.	A stable, unchanging position inside the current function's memory.
	Changes	Changes continuously with **every instruction**, loop, or jump.	Changes when **items are pushed or popped** within a function.	Stays completely **stationary for duration of function**.
	Name	Called PC across architectures.	SP (or R13 / X31).	FP in ARM (X29), BP in x86 (EBP/RBP).

Instruction Categories

- **Integer Arithmetic** (our focus in this presentation)
  - Arithmetic, logic, and shift instructions
- **Data Transfer**
  - Load and store instructions that access memory
  - Data movement and conversions
- **Jump and Branch**
  - Flow-control instructions that alter the sequence

And more to come later …

- **Floating Point Arithmetic**
  - Instructions that operate on floating-point registers
- **Miscellaneous**
  - Instructions that transfer control to/from exception handlers
  - Memory management instructions

MIPS Assembly Programming

- There are three main types of statements that can be used in assembly language, where each statement appears on a separate line:

1. **Assembler directives:** These provide information to the assembler while translating a program.
    - Directives are used to **define segments** and **allocate space for variables in memory**.
    - An assembler directive always starts with a dot.
    - Example directives:
        - **.data** Defines the program's data segment, containing the program's variables.
        - **.text** Defines the code segment of the program, containing the instructions.
        - **.globl** Defines a symbol as global that can be referenced from other files.

MIPS Assembly Programming

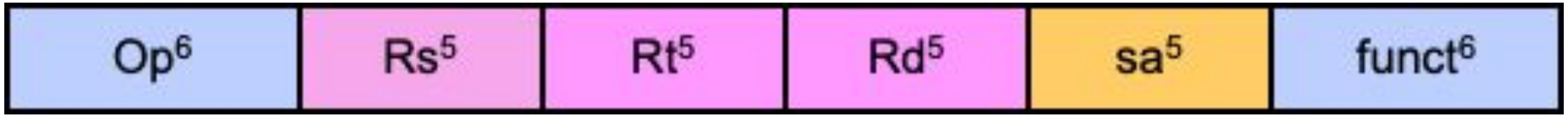
2. **Executable Instructions:** These generate machine code for the processor to execute at runtime.
   - Instructions tell the processor what to do.

3. **Pseudo-Instructions and Macros:**
   Translated by the assembler into real instructions.
   - These simplify the programmer's task.
   - Example:

     move —> **addu $rd, $rs, $zero** (**Add the source register to register $zero**)

     (So, add is equivalent to move, because 5+0 = 5).

R-Type Instruction Format



- Op: operation code (opcode)
  - Specifies the operation of the instruction
  - Also specifies the format of the instruction
- funct: function code – extends the opcode
  - Up to 26 = 64 functions can be defined for the same opcode
  - MIPS uses opcode 0 to define many R-type instructions
- Three Register Operands (common to many instructions)
  - Rs, Rt: first and second source operands
  - Rd: destination operand
  - sa: the **shift amount** used by **shift instructions**

R-Type Integer Add and Subtract

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func
add $t1, $t2, $t3	$t1 = $t2 + $t3	0	$t2	$t3	$t1	0	20
addu $t1, $t2, $t3	$t1 = $t2 + $t3	0	$t2	$t3	$t1	0	21
sub $t1, $t2, $t3	$t1 = $t2 – $t3	0	$t2	$t3	$t1	0	22
subu $t1, $t2, $t3	$t1 = $t2 – $t3	0	$t2	$t3	$t1	0	23

- add, sub: arithmetic overflow causes an exception
  - In case of overflow, the result is not written to the destination register.
- addu, subu: arithmetic overflow is ignored
- addu, subu: compute the same result as add, sub
- Many programming languages ignore overflow
  - The + operator is translated into addu
  - The – operator is translated into subu

Assembly Code Template

- A MIPS assembly language program template is shown here:

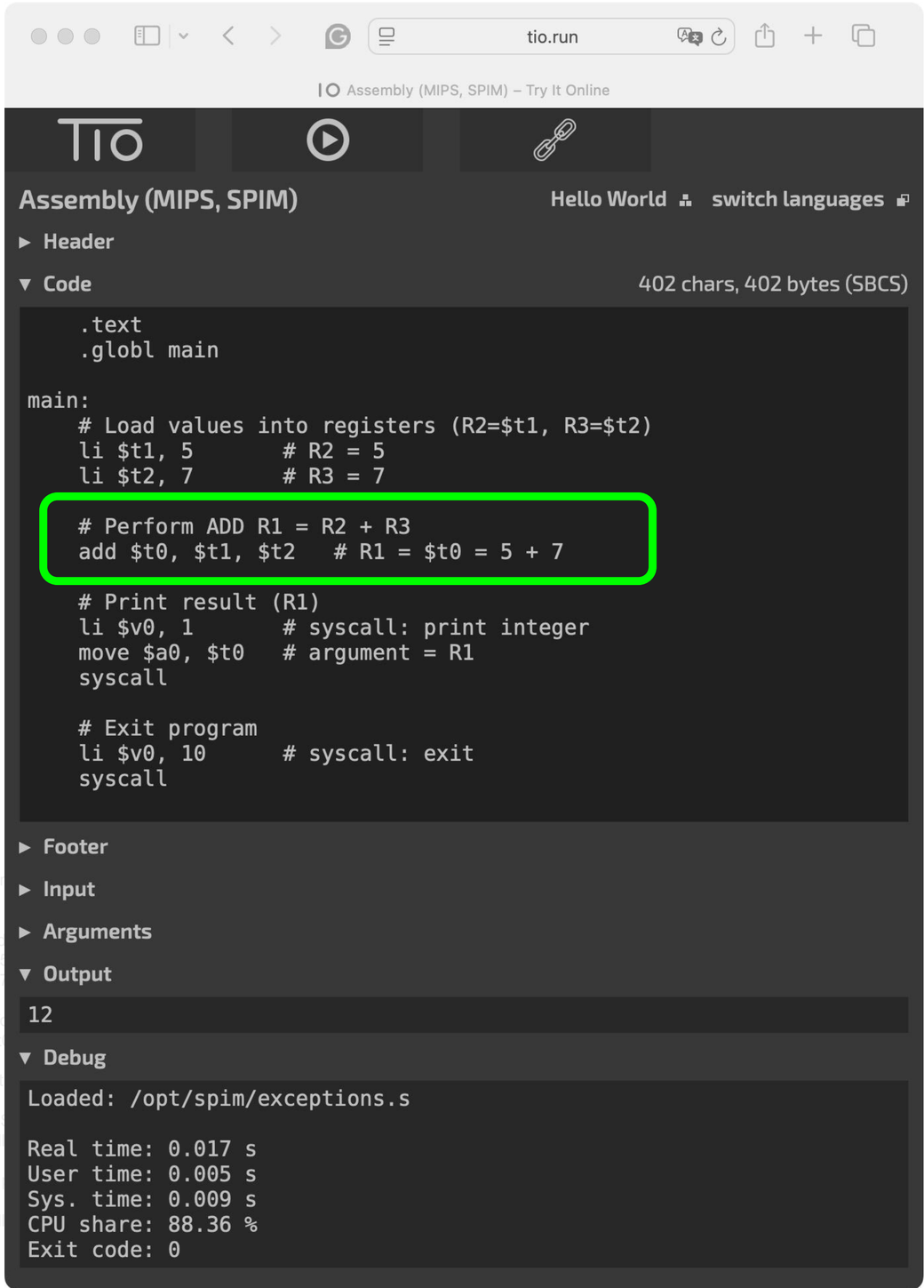
```
# Title:
# Author:
# Date:
# Description:
# Input:
# Output:

#################### Data segment ######################
.data
. . .

#################### Code segment ######################
.text
.globl main
main: # main function entry
    . . .
    li $v0, 10
    syscall     # system call to exit program
```

MIPS

- MIPS uses instructions like **ADD** R1, R2, R3 directly.
- This way:
  - $t0 acts as R1
  - $t1 acts as R2
  - $t2 acts as R3
- We can change numbers in li $t1, ... and li $t2, ... to test.
- https://tio.run/# then choose the language "Assembly (MIPS, SPIM)"

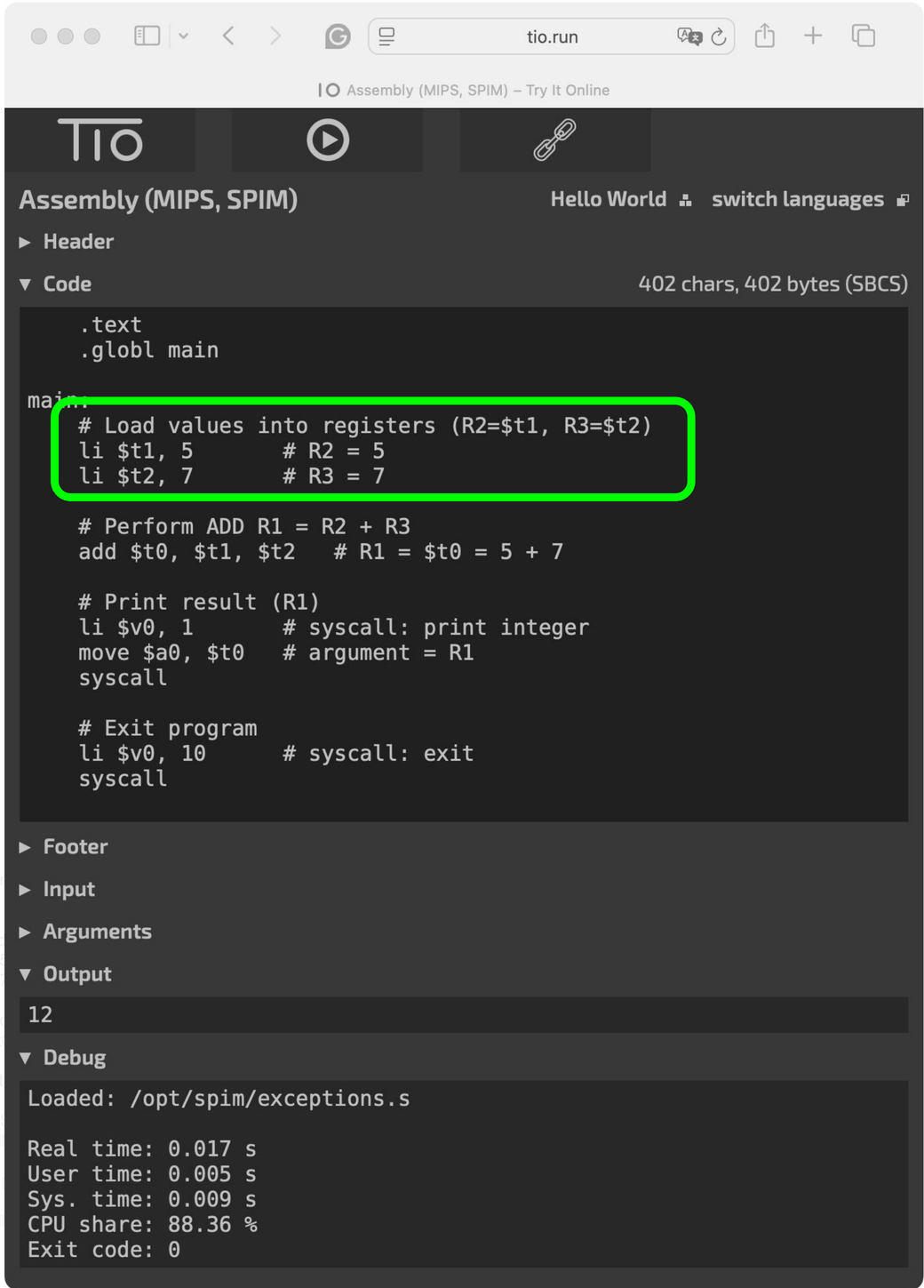


MIPS

- Initialization (Register Loading)

```
li $t1, 5       # R2 = $t1 = 5
li $t2, 7       # R3 = $t2 = 7
```

- li (Load Immediate): Loads a constant value into a register.
- $t1 (temporary register 1) is set to 5.
- $t2 (temporary register 2) is set to 7.
- Note:
  - In the comments, R2 and R3 refer to the logical registers in your original question (mapped to $t1 and $t2 in MIPS).

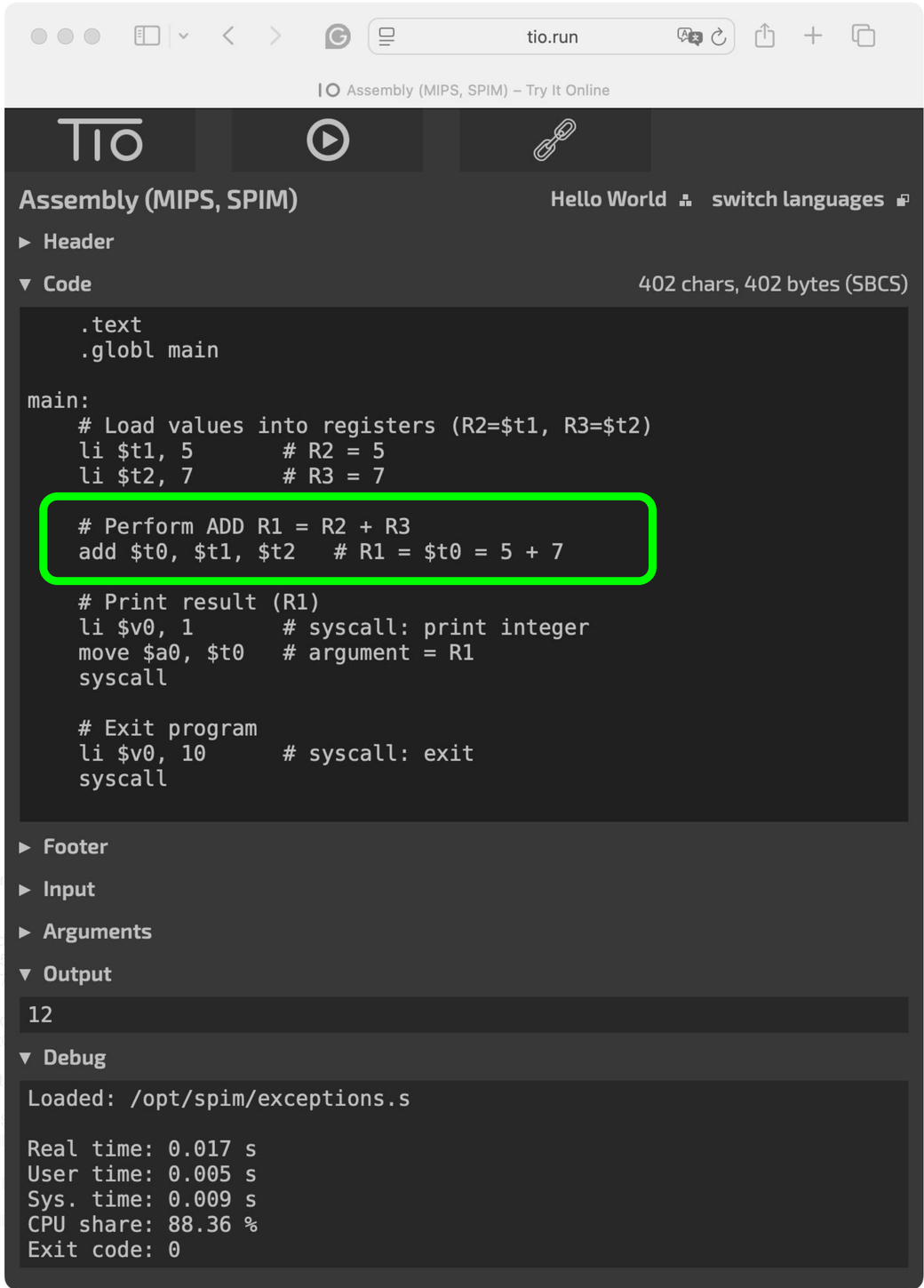


MIPS

- Addition Operation

  add $t0, $t1, $t2 # R1 = $t0 = 5 + 7

- add: Adds the values of $t1 (5) and $t2 (7), storing the result in $t0 (temporary register 0).
- Result: $t0 = 12 (which is referred to as R1 in the comments).

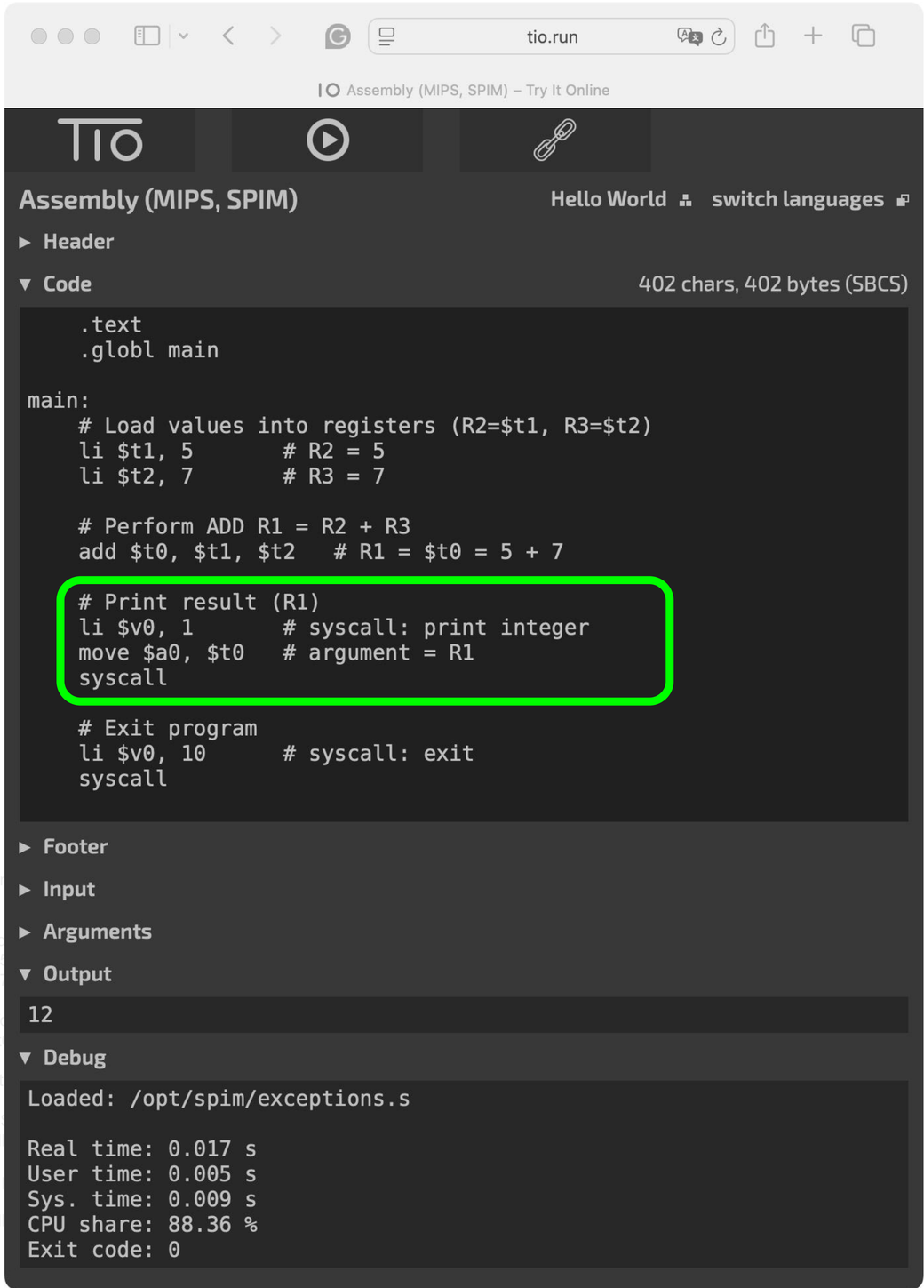


MIPS

- Print the Result

```
li $v0, 1          # Prepare to print an integer
move $a0, $t0      # Set argument = $t0 (R1 = 12)
syscall            # Execute the print
```

- li $v0, 1: Loads the syscall code 1 into $v0 (MIPS uses $v0 to determine the syscall type).
  - 1 = “print integer” service.
- move $a0, $t0: Copies the value of $t0 (12) to $a0 (the register for syscall arguments).
- syscall: Triggers the syscall, printing the integer (12) to the console.

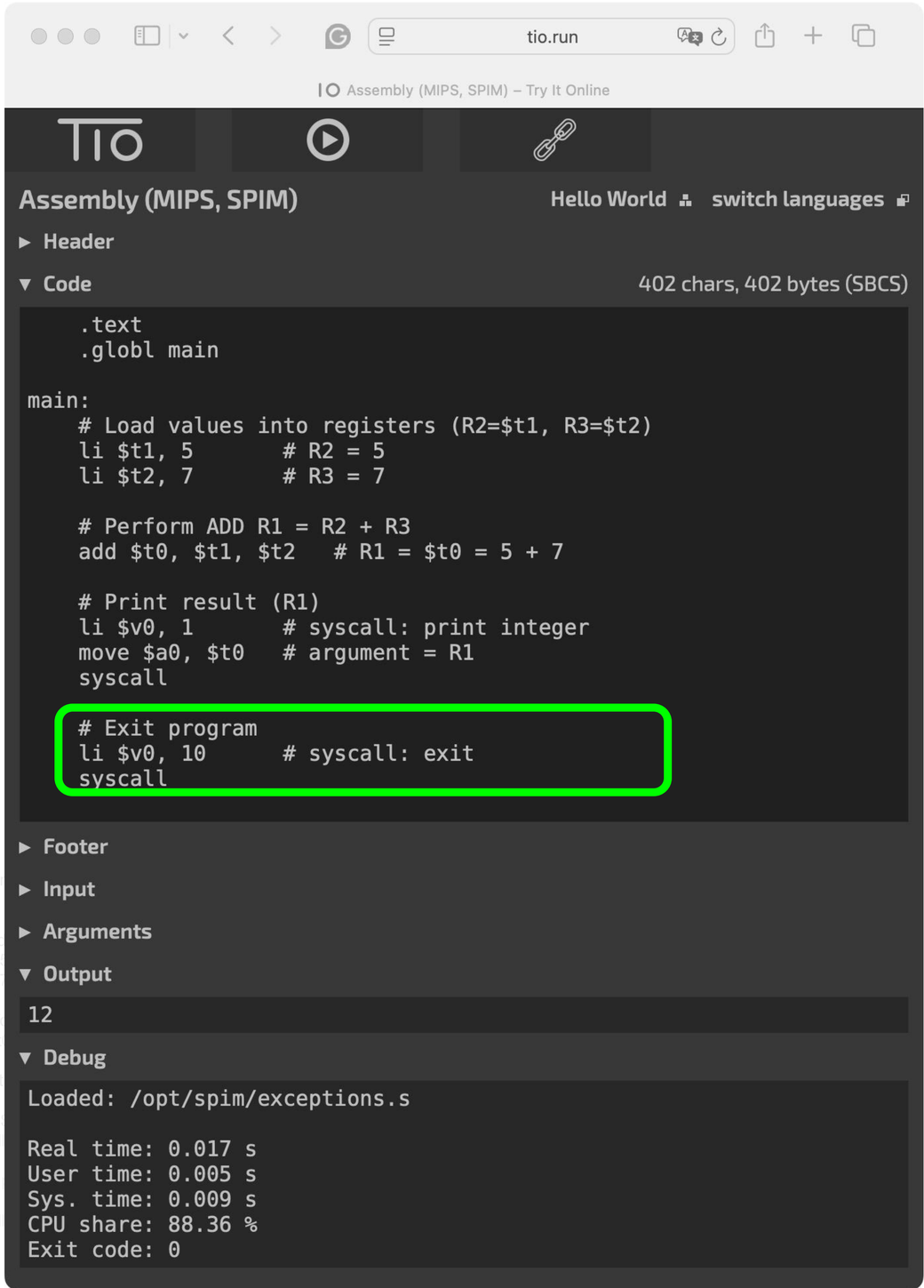


MIPS

- Exit the Program

```
li $v0, 10      # Prepare to exit
syscall         # Terminate the program
```

- li $v0, 10: Loads the syscall code 10 into $v0.
  - 10 = “exit” service.
- syscall: Ends the program execution.



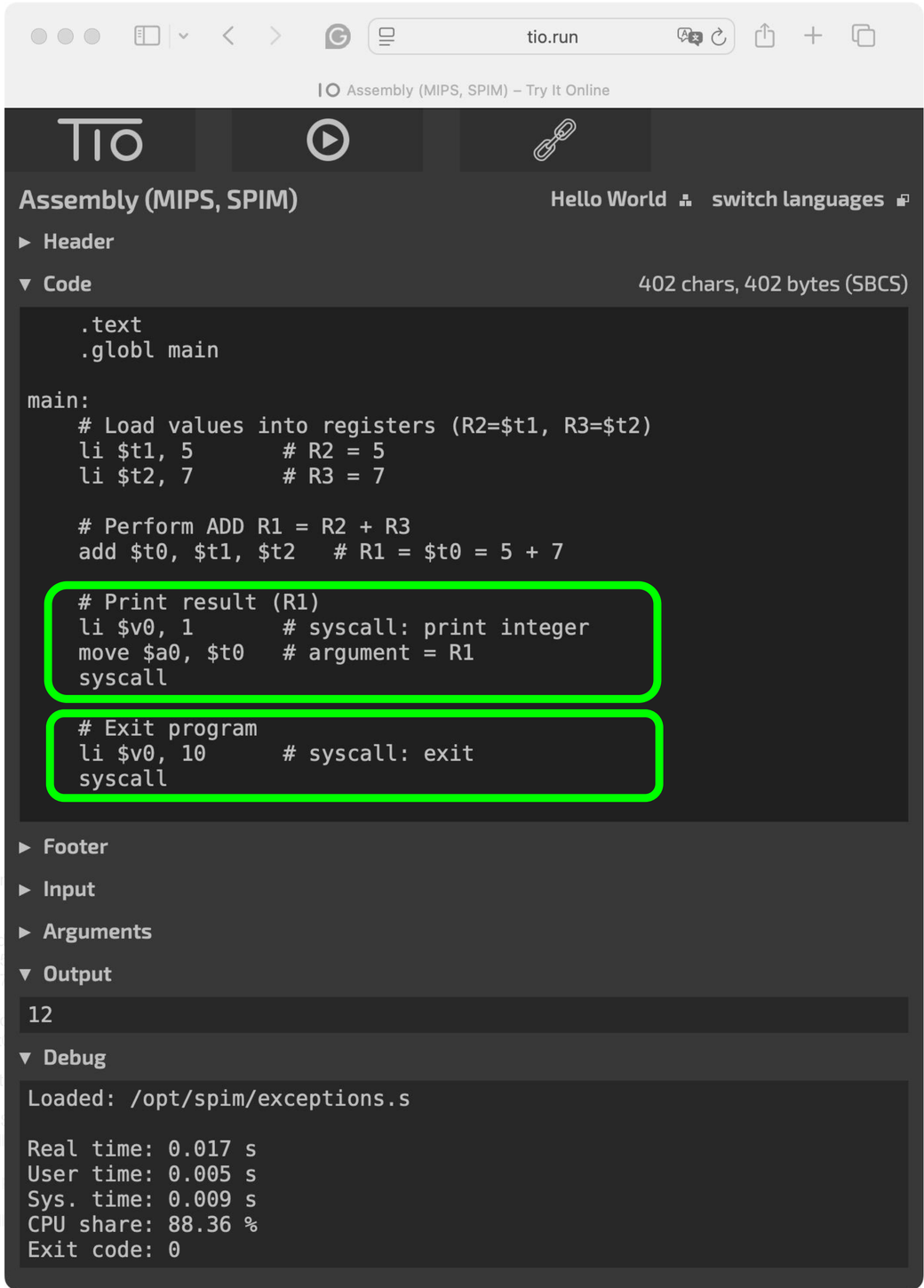
MIPS

MIPS Register Conventions:

- **$t0–$t9**: Temporary registers (caller-saved).
- **$v0**: Holds syscall numbers and return values.
- **$a0–$a3**: Store arguments for syscalls/ functions.

Syscalls:

- MIPS uses **syscall** to interact with the OS, e.g.,
  - Program termination = **10**
  - Print Integer = **1**
  - I/O, = ….



Code to copy/paste



Key Points: Assembly Programming on MIPS

- **MIPS: Microprocessor without Interlocked Pipeline Stages**
  - 32 general-purpose registers, all 32-bit wide
- **Assembly statements: directives, instructions, macros**
  - Template: .data section, .text section, main label
- **Key instructions: li, add, sub, syscall**
  - $t0-$t9 (temp), $s0-$s7 (saved), $v0-$v1 (return)
- **R-Type format: opcode, rs, rt, rd, shamt, funct**
  - add/sub with arithmetic overflow cause exceptions

More next class on this:

- **Unsigned integers: range 0 to 2^n - 1**
  - Carry = 1 signals unsigned result out of range
- **Signed (2's complement): range -2^(n-1) to 2^(n-1) - 1**
  - Positive overflow wraps to most negative
  - Negative overflow wraps to max positive
- **Carry for unsigned sums; Overflow for signed sums**

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025

Mahidol University